



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

KOMPUTERALGEBRA TANSZÉK

Zero Knowledge Proof (ZKP) alapú
hitelesítési rendszer implementálása TOTP
kétlépcsős azonosítással

Témavezető:

Nagy Ádám

Mérnökhallgató

Szerző:

Szabó Bence Dániel

Programtervező Informatikus BSc

Budapest, 2026

Nyilatkozat

Alulírott **Szabó Bence Dániel** (Neptun kód: **EL1QUI**), az Eötvös Loránd Tudományegyetem Informatikai Karának hallgatója kijelentem, hogy ezt a szakdolgozatot saját magam készítettem, és abban csak a megadott forrásokat használtam fel. Minden olyan részt, amelyet szó szerint vagy azonos tartalommal, de átfogalmazva más forrásból átvettem, egyértelműen, a forrás megadásával megjelöltem.

A dolgozat bírálója által hátrányos értékeléssel nem járó exkluzív elektronikus hozzáférést a benyújtástól számított két évig nem kérem.

Budapest, 2026. április 30

Szabó Bence Dániel s.k.
hallgató aláírása

Összefoglaló

A dolgozat egy zéró tudású bizonyításon (ZKP) alapuló, biztonságos valós idejű csevegőalkalmazás tervezését és megvalósítását mutatja be. A rendszer a Schnorr-protokollt alkalmazza, amely lehetővé teszi, hogy a felhasználó a jelszó felfedése nélkül bizonyítsa személyazonosságát — így a szerveren soha nem tárolódik jelszó. A hibrid architektúra REST API-t (hitelesítés, TOTP 2FA) és WebSocket-et (valós idejű üzenetküldés) ötvöztet, Go szerver és JavaScript kliens implementációval.

Kulcsszavak: zéró tudású bizonyítás, Schnorr-protokoll, WebSocket, Go, kétfaktoros azonosítás

Abstract

This thesis presents the design and implementation of a secure real-time chat application based on Zero Knowledge Proof (ZKP) authentication. The system employs the Schnorr protocol, enabling users to prove their identity without ever transmitting passwords to the server. The hybrid architecture combines a REST API (authentication, TOTP 2FA) with WebSocket (real-time messaging), implemented with a Go server and a JavaScript client.

Keywords: zero knowledge proof, Schnorr protocol, WebSocket, Go, two-factor authentication

Tartalomjegyzék

Nyilatkozat	1
Összefoglaló	2
Abstract	3
Rövidítésjegyzék	8
1. Bevezetés	9
1.1. Motiváció — A jelszó-alapú hitelesítés problémái	9
1.2. A dolgozat célja	9
1.3. A dolgozat felépítése	10
I. Felhasználói dokumentáció	11
2. A zéró tudású bizonyítás alapgondolata	12
2.1. Mi az a zéró tudású bizonyítás?	12
2.2. Hogyan alkalmazható a ZKP hitelesítésre?	14
2.3. Miért biztonságosabb, mint a jelszóküldés?	14
3. Az alkalmazás bemutatása	16
3.1. Rendszerkövetelmények és első indítás	16
3.1.1. Üzemeltetői oldal (backend)	16
3.1.2. Felhasználói oldal (frontend)	17
3.2. Funkcionalitás áttekintése	17
3.3. Regisztráció	18
3.4. Bejelentkezés	18
3.5. Kétfaktoros azonosítás (TOTP) beállítása és használata	19
3.6. Csevegés — valós idejű üzenetküldés	20

3.7.	Jelszóvisszaállítás recovery kulccsal	21
3.8.	Hibaüzenetek és teendők	22
II. Fejlesztői dokumentáció		23
4. Elméleti alapok		24
4.1.	Véges testek és moduláris aritmetika	24
4.2.	Ciklikus csoportok és generátorelemek	25
4.3.	A Diszkrét Logaritmus Probléma (DLP) és annak aszimmetriája . . .	25
4.4.	A Schnorr-protokoll	26
4.4.1.	A protokoll áttekintése	26
4.4.2.	Rendszerparaméterek	26
4.4.3.	A protokoll lépései	27
4.4.4.	Ellenőrzés (Verification)	27
4.4.5.	Biztonsági tulajdonságok	28
4.5.	Szakirodalmi áttekintés — ZKP-alapú hitelesítés	28
4.5.1.	Hagyományos jelszó-alapú hitelesítés sebezhetőségei	28
4.5.2.	ZKP hitelesítési architektúrák az irodalomban	29
4.5.3.	PAKE protokollcsalád	30
4.5.4.	ZKP-alapú jelszó-visszaállítás és kulcskezelés	31
4.5.5.	A Schnorr-protokoll választásának indoklása	32
5. Használt technológiák		34
5.1.	Go 1.24	34
5.1.1.	gorilla/websocket	34
5.1.2.	chi/v5 HTTP router	35
5.1.3.	SQLite (modernnc.org/sqlite, tiszta Go)	35
5.1.4.	JWT (HMAC-SHA256)	35
5.1.5.	pquerna/otp (TOTP)	36
5.2.	Kliens-oldali technológiák	36
6. Komponensek felépítése és kapcsolataik		37
6.1.	Architektúra áttekintés	37
6.2.	Adatmodell	38
6.3.	A hibrid kommunikációs réteg (REST + WebSocket)	40

6.3.1.	A HTTP REST API szerepe	41
6.3.2.	A WebSocket csatorna	41
6.4.	Biztonsági megfontolások	43
6.5.	Telepítési architektúra	43
6.6.	Modulstruktúra	44
6.7.	Rendszerdiagramok	45
7.	Az egyes komponensek működése	46
7.1.	ZKP regisztráció és bejelentkezés	46
7.1.1.	Regisztráció (US-01)	46
7.1.2.	Bejelentkezés — Challenge (US-02)	47
7.1.3.	Bejelentkezés — Proof Verify (US-03)	47
7.1.4.	Hibakezelés és biztonság (US-15)	48
7.1.5.	A Schnorr-ellenőrzés implementációja	49
7.2.	TOTP kétfaktoros azonosítás	49
7.2.1.	TOTP beállítás (US-04)	51
7.2.2.	TOTP ellenőrzés (US-05)	51
7.2.3.	TOTP kikapcsolás (US-14)	52
7.3.	Chat funkció	52
7.3.1.	Hub architektúra	52
7.3.2.	WebSocket kapcsolat és hitelesítés (US-16)	53
7.3.3.	Felhasználók listázása (US-11)	54
7.3.4.	Üzenet küldése és fogadása (US-06, US-07)	54
7.3.5.	Korábbi üzenetek (US-12)	55
7.3.6.	Offline üzenetek (US-17)	55
7.3.7.	Kliens-oldali chat modul	56
7.4.	Jelszóvisszaállítás és session kezelés	56
7.4.1.	Recovery key generálás (US-13)	56
7.4.2.	Jelszóvisszaállítás (US-08, US-09)	57
7.4.3.	Kijelentkezés (US-10)	58
7.4.4.	Token lejárat és session kezelés (US-18)	58
7.5.	Alternatív telepítés: Docker konténer	58
7.5.1.	Image felépítése	59
7.5.2.	Perzisztencia és konfiguráció	60

7.5.3.	Orkesztráció Docker Compose-szal	61
7.5.4.	Futtatási workflow	62
8.	Tesztelés	63
8.1.	A tesztpiramis alkalmazása	63
8.2.	1. réteg: Egységtesztek	64
8.2.1.	Mock repository és service-szintű hibaág-tesztek	65
8.2.2.	Crypto egységtesztek	66
8.2.3.	Middleware és envelope tesztek	67
8.3.	2. réteg: Integrációs tesztek	68
8.3.1.	ZKP autentikációs folyamat	68
8.3.2.	Repository és HTTP handler tesztek	71
8.3.3.	WebSocket tesztek	75
8.4.	3. réteg: End-to-end tesztek	76
8.5.	Tesztelési összefoglaló	78
9.	Összefoglalás	79
9.1.	Elért eredmények	79
9.2.	Korlátok	80
9.3.	Jövőbeli fejlesztési lehetőségek	80

Rövidítésjegyzék

2FA	Two-Factor Authentication — kétfaktoros azonosítás
API	Application Programming Interface — alkalmazásprogramozási felület
BigInt	Tetszőleges pontosságú egész szám típus (JavaScript)
CORS	Cross-Origin Resource Sharing — kereszt-eredetű erőforrás-megosztás
DLP	Discrete Logarithm Problem — diszkrét logaritmus probléma
E2EE	End-to-End Encryption — végpontok közötti titkosítás
HMAC	Hash-based Message Authentication Code
HOTP	HMAC-Based One-Time Password
IIFE	Immediately Invoked Function Expression (JavaScript minta)
JWT	JSON Web Token — JSON alapú hitelesítési token
KDF	Key Derivation Function — kulcsszármaztató függvény
NIZK	Non-Interactive Zero-Knowledge — nem-interaktív zéró tudású bizonyítás
PAKE	Password-Authenticated Key Exchange
PBKDF2	Password-Based Key Derivation Function 2
REST	Representational State Transfer
TOTP	Time-Based One-Time Password — időalapú egyszeri jelszó
TTL	Time To Live — élettartam
WS	WebSocket
XSS	Cross-Site Scripting
ZKP	Zero Knowledge Proof — zéró tudású bizonyítás

1. fejezet

Bevezetés

1.1. Motiváció — A jelszó-alapú hitelesítés problémái

A webes szolgáltatások túlnyomó többsége ma is felhasználónév–jelszó párra épülő hitelesítést használ. A módszer egyszerű, de két visszatérő gyenge pontja van. Az egyik a felhasználó oldalán: a gyenge vagy több oldalon újrafelhasznált jelszavakat egy erős szerveroldali hash-algoritmus sem védi meg. A másik a szerver oldalán: a bejelentkezés pillanatában a jelszó legalább rövid időre megjelenik a szerveren, és egy sikeres szervertörés során közvetlenül kikerülhet.

Erre a problémára kínálnak megoldást a *zéró tudású bizonyítások* (Zero Knowledge Proofs, ZKP) [1]: olyan kriptográfiai eljárások, amelyekkel a felhasználó matematikailag igazolja, hogy ismeri a jelszavát, anélkül hogy azt bármilyen formában elküldené. A szerver így úgy tud meggyőződni a felhasználó személyazonosságáról, hogy magát a jelszót soha nem látja.

1.2. A dolgozat célja

A dolgozat egy működő csevegőalkalmazás megtervezését és megvalósítását dokumentálja. A rendszer a Schnorr-protokollon [2] alapuló zéró tudású bizonyítással azonosítja a felhasználókat, és az alábbi fő funkciókat nyújtja:

- **ZKP-alapú regisztráció és bejelentkezés** — a szerveren soha nem tárolódik jelszó, kizárólag egy nyilvános kriptográfiai kulcs.

- **TOTP kétfaktoros azonosítás** — az RFC 6238 szabvány szerinti időalapú egyszeri jelszó [3], amely további védelmi réteget biztosít.
- **Valós idejű csevegés** — WebSocket alapú, alacsony késleltetésű üzenetküldés jelenlétjelzéssel.
- **Jelszóvisszaállítás recovery kulccsal** — a jelszóvisszaállítás is ZKP-alapú, így a kliens a recovery kulcsot sem fed fel.

1.3. A dolgozat felépítése

A szakdolgozat felhasználói és fejlesztői dokumentációra oszlik:

I. rész — Felhasználói dokumentáció (1–3. fejezet): Ez a rész az olvasónak nem feltételez mély technikai háttérrel. Bemutatja a zéró tudású bizonyítás alapgon-
dolatát közérthető formában, majd ismerteti az alkalmazás funkcióit a felhasználó
szemszögéből.

- A 2. fejezet a zéró tudású bizonyítás intuitív magyarázatát adja, analógiákkal szemléltetve a működést.
- Ezt követően a 3. fejezetben a felhasználó szemszögéből tekintem át a rendszer funkcióit: regisztráció, bejelentkezés, csevegés és jelszóvisszaállítás.

II. rész — Fejlesztői dokumentáció (4–9. fejezet): Ez a rész a rendszer technikai részleteivel foglalkozik.

- A 4. fejezet a matematikai alapokat és a Schnorr-protokollt tárgyalja, majd szakirodalmi áttekintéssel helyezi kontextusba a ZKP-alapú hitelesítést.
- A felhasznált technológiákat és a választás indoklását a 5. fejezet ismerteti.
- A 6. fejezet a rendszer architektúráját és komponenseit mutatja be, a 7. fejezet pedig az egyes komponensek részletes implementációját írja le.
- Végül a 8. fejezet a tesztelési stratégiát és eredményeket tartalmazza, a 9. fejezet pedig összegzi a munkát és a továbbfejlesztési irányokat.

I. rész

Felhasználói dokumentáció

2. fejezet

A zéró tudású bizonyítás alapgondolata

A dolgozatban bemutatott alkalmazás biztonsága egyetlen kriptográfiai alapelvre épül: a zéró tudású bizonyításra (Zero Knowledge Proof, ZKP). A fejezet a fogalom intuitív magyarázatát adja, technikai részletek nélkül.

2.1. Mi az a zéró tudású bizonyítás?

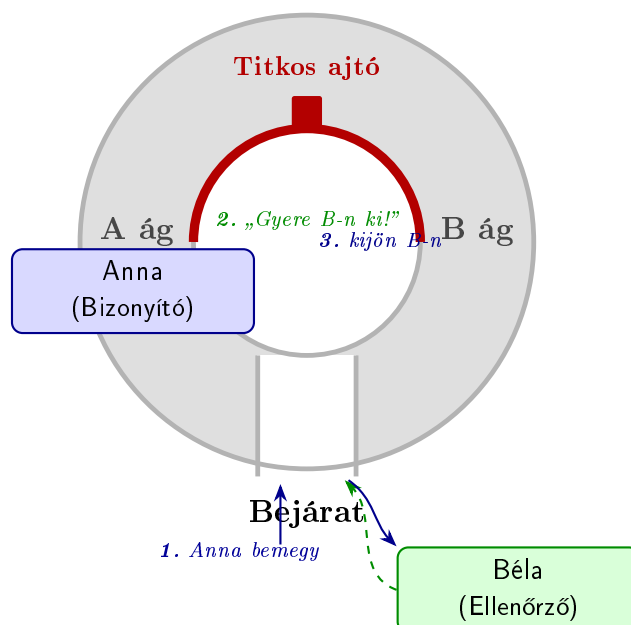
A zéró tudású bizonyítás egy olyan módszer, amellyel valaki be tudja bizonyítani, hogy ismer egy titkot, anélkül hogy magát a titkot felfedné. Első hallásra ez paradoxonnak tűnhet: hogyan győzhet meg valakit arról, hogy tud valamit, ha nem árulja el, mit tud? A válasz a matematikában rejlik: a bizonyító és az ellenőrző fél egy kérdés-válasz párbeszédet folytat, amelynek végén az ellenőrző meggyőződhet az állítás igazságáról, de a titokból semmilyen információt nem szerez.

Ali baba barlangja. Az egyik legismertebb analógia a ZKP működésének szemléltetésére az úgynevezett *Ali baba barlangja* gondolkísérlet. Képzeljünk el egy körkörös barlangot, amelynek a bejáratánál az út kétfelé ágazik (A és B irány), majd a hátsó részen egy zárt ajtó választja el a két ágot. Az ajtó csak egy titkos varázsszóval nyitható.

A bizonyító (Anna) azt állítja, hogy ismeri a varázsszót. Az ellenőrző (Béla) meg akar győződni erről, anélkül hogy meghallaná a szót. A játék a következőképpen zajlik:

1. Béla a barlang bejáratánál marad, nem lát be.
2. Anna besétál a barlangba, és véletlenszerűen az A vagy B ágot választja.
3. Béla bekiált, hogy melyik ágon szeretné látni kijönni Annát (A vagy B).
4. Ha Anna tud átjutni az ajtón (mert ismeri a varázsszót), mindig teljesíteni tudja Béla kérését. Ha nem ismeri a szót, csak 50% eséllyel találja el véletlenül a helyes oldalt.
5. Ha ezt a játékot sokszor megismétlik, és Anna *minden* alkalommal a kért oldalon jön ki, Béla szinte biztossá válhat abban, hogy Anna ismeri a varázsszót, miközben ő maga semmit nem tudott meg a szó tartalmáról.

Ez a gondolkísérlet a ZKP három alapvető tulajdonságát szemlélteti. A *teljeség* azt mondja ki, hogy Anna, ha valóban ismeri a titkot, mindig meg tudja győzni Bélát. A *megbízhatóság* az ellenkező irányból fogja meg a kérdést: a titkot nem ismerő Anna csak 50% eséllyel találja el a helyes oldalt, így a játék ismétlésével a csalás valószínűsége tetszőlegesen kicsivé tehető (bár az 1-et sosem éri el). Végül a *zéró tudás* tulajdonság garantálja, hogy Béla a játék végén semmit nem tud meg a varázsszóról, csak azt, hogy Anna ismeri.



2.1. ábra. Az Ali Baba barlangja — felülnézeti vázlat. **1.** Anna (Bizonyító) véletlenszerűen választ egy ágot és bemegy, **2.** Béla (Ellenőrző) bekiáltja a kívánt kijáratú ágot, **3.** Anna, ismerve a varázsszót, mindig a kért oldalon lép ki.

2.2. Hogyan alkalmazható a ZKP hitelesítésre?

A hagyományos bejelentkezési rendszerekben a felhasználó elküldi a jelszavát a szerver felé, amely összehasonlítja a tárolt változattal. Ez azt jelenti, hogy a jelszó (ha csak egy pillanatra is) megjelenik a szerveren. A ZKP-alapú hitelesítés ezt a lépést kiváltja: a felhasználó jelszava helyett egy matematikai bizonyítást küld, amely igazolja, hogy ismeri a jelszót.

A folyamat a gyakorlatban a következőképpen zajlik:

1. **Regisztráció:** A felhasználó kiválaszt egy jelszót, de nem küldi el a szervernek. Helyette a böngésző a jelszóból egy matematikai kulcspárt számol: egy *titkos kulcsot* (amelyet csak a felhasználó ismer) és egy *nyilvános kulcsot* (amelyet a szerver tárol). A nyilvános kulcsból a jelszó nem fejthető vissza.
2. **Bejelentkezés:** A szerver egy véletlenszerű kihívást („feladványt”) küld a felhasználónak. A felhasználó böngészője a titkos kulcs segítségével kiszámítja a helyes választ, és visszaküldi. A szerver a nyilvános kulccsal ellenőrzi, hogy a válasz helyes-e.
3. **Eredmény:** Ha a válasz helyes, a szerver elfogadja a bejelentkezést. Ha nem, elutasítja. A jelszó (és a belőle származó titkos kulcs) egyetlen pillanatra sem jelent meg a szerveren.

Ez a folyamat megfelel a barlangos analógia logikájának: a szerver feltesz egy kérdést („melyik oldalon gyere ki?”), a felhasználó válaszol (átmegy az ajtón), és a szerver meggyőződik az eredményről, anélkül, hogy megtudná a varázsszót (jelszót).

2.3. Miért biztonságosabb, mint a jelszóküldés?

A ZKP-alapú hitelesítés több szempontból is erősebb védelmet nyújt, mint a hagyományos módszer:

- **A szerver nem látja a jelszót.** Még ha egy támadó feltöri a szervert, sem talál jelszavakat vagy jelszó-hasheket, kizárólag nyilvános kulcsokat, amelyekből a jelszó nem állítható helyre.

- **A hálózati forgalom lehallgatása sem segít.** A kommunikáció során nem a jelszó utazik, hanem egy egyszeri matematikai bizonyítás, amely egy másik bejelentkezéshez nem használható fel újra.
- **Matematikailag bizonyított biztonság.** A ZKP biztonságát nehéz matematikai problémák garantálják: formálisan bizonyítható, hogy hatékony algoritmus nem képes a titkot kinyerni a bizonyítási folyamatból. A ZKP mint általános módszer különböző nehéz problémákra építhető; a dolgozatban bemutatott Schnorr-protokoll konkrétan a diszkrét logaritmus problémát (DLP) alkalmazza, amely az egyik legjobban tanulmányozott ilyen nehézségi feltételezés.

A ZKP-alapú hitelesítés tehát a titok megosztása helyett csak a titok *ismeretét* bizonyítja.

3. fejezet

Az alkalmazás bemutatása

A csevegőalkalmazás felhasználói útja a regisztrációnál kezdődik, és a valós idejű üzenetváltásig tart. A fejezet ezt az utat járja végig: minden alfejezet egy-egy funkciót mutat be a felhasználó szemszögéből.

3.1. Rendszerkövetelmények és első indítás

3.1.1. Üzemeltetői oldal (backend)

A szerver üzemeltetéséhez az alábbi szoftverkörnyezet szükséges:

- **Go 1.24** vagy újabb verzió. A szerver operációs rendszertől függetlenül fut (Linux, macOS, Windows).
- **make** build eszköz (opcionális; a `go run ./cmd/server` paranccsal helyettesíthető).
- **Hálózat:** A szerver alapértelmezetten a `localhost:8080` porton hallgat; külső eléréshez a megfelelő tűzfalszabály szükséges.

A szerver indítása:

1. A forráskód könyvtárában futtatni kell a `go mod tidy` parancsot a függőségek letöltéséhez.
2. A szerver a `make run` (vagy `go run ./cmd/server`) paranccsal indítható el.

3. Az első indításkor a szerver automatikusan előállítja a 2048 bites Schnorr-csoportparamétereket (ez akár egy percig is eltarthat). A paraméterek a `params.json` fájlba mentődnek, és a következő indításkor onnan töltődnek be.

3.1.2. Felhasználói oldal (frontend)

A webalkalmazás eléréséhez kizárólag egy modern böngésző szükséges; semmilyen telepítés nem szükséges a felhasználó gépén. Támogatott böngészők (a JavaScript BigInt és Web Crypto API-t igényelik):

- Google Chrome 67+, Mozilla Firefox 68+, Microsoft Edge 79+, Safari 14+.

Az alkalmazás megnyitása:

1. A böngészőben meg kell nyitni a `http://localhost:8080` címet (vagy a szerver publikus elérési útját).
2. A rendszer külső függőség (adatbázis-szerver, Docker stb.) nélkül azonnal használható: regisztrálás után a chat funkció rögtön elérhető.

3.2. Funkcionalitás áttekintése

Az alkalmazás egy böngészőben futó valós idejű csevegőrendszer, amely az alábbi fő funkciókat kínálja:

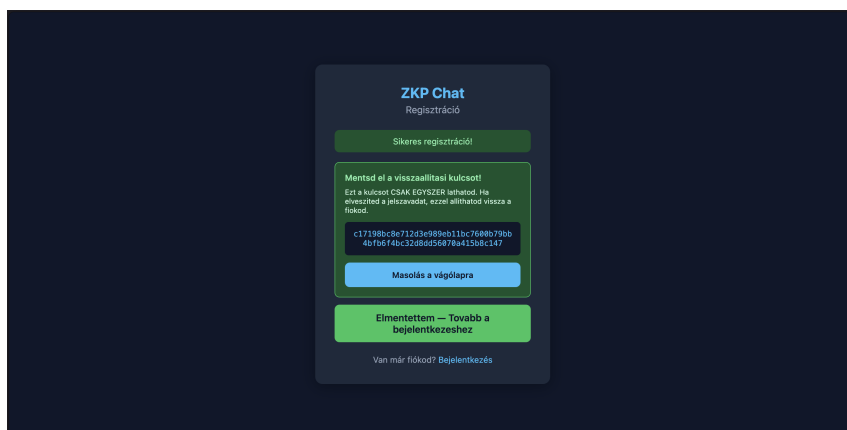
- **Regisztráció** — felhasználói fiók létrehozása, recovery kulcs generálásával.
- **Bejelentkezés** — zéró tudású bizonyításon alapuló, biztonságos belépés.
- **Kétfaktoros azonosítás (2FA)** — opcionális, időalapú egyszeri jelszó (TOTP) a még magasabb biztonság érdekében.
- **Valós idejű csevegés** — azonnali üzenetküldés más felhasználóknak, online/offline jelzéssel.
- **Jelszóvisszaállítás** — recovery kulccsal végrehajtható biztonságos jelszócsere.

3.3. Regisztráció

A regisztrációs folyamat a felhasználó számára egyszerű és ismerős:

1. A felhasználó megadja a kívánt **felhasználónevet** és **jelszót** a regisztrációs űrlapon.
2. A rendszer rövid feldolgozás után visszajelez, hogy a regisztráció sikeres volt.
3. Megjelenik egy **recovery kulcs**: egy hosszú, véletlenszerű karaktersorozat, amely a jelszó elvesztése esetén szolgál biztonsági mentésként.
4. A felhasználónak el kell mentenie ezt a kulcsot egy biztonságos helyre (pl. jelszókezelő, papír), mivel ez az egyetlen módja a fiók visszaállításának, ha elfelejti a jelszavát. A rendszer ezt a kulcsot nem tárolja és később nem jeleníti meg újra.

A háttérben a böngésző a jelszóból kriptográfiai kulcsokat számol, és kizárólag a nyilvános komponenst küldi el a szervernek. A felhasználó ebből semmit nem észlel, a folyamat ugyanolyan gyors és egyszerű, mint bármely más webes regisztráció.



3.1. ábra. Sikeres regisztráció — a rendszer megjeleníti a recovery kulcsot, amelyet a felhasználónak el kell mentenie

3.4. Bejelentkezés

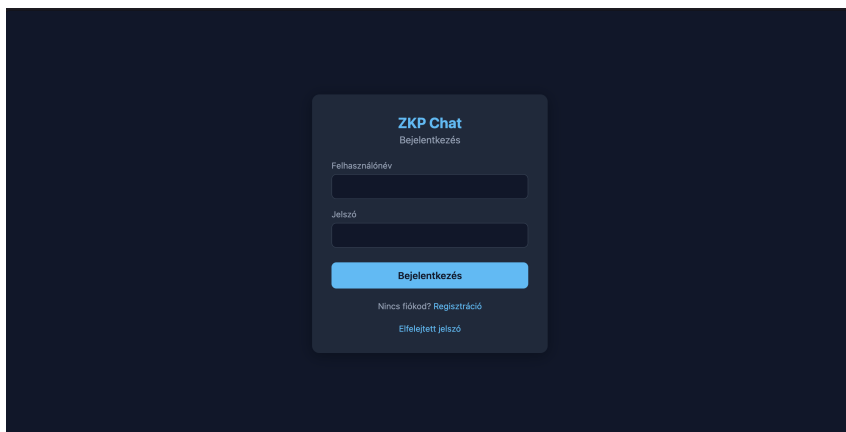
A bejelentkezés a következő képpen történik:

1. A felhasználó beírja a **felhasználónevét** és **jelszavát**.
2. A „Bejelentkezés” gombra kattint.

3. A böngésző automatikusan végrehajtja a háttérben a kriptográfiai bizonyítást, ami mindössze egy-két másodpercet vesz igénybe.
4. Sikeres hitelesítés esetén a felhasználó közvetlenül a csevegőfelületre kerül.

A felhasználó számára láthatatlan, de a háttérben egy háromlépéses matematikai párbeszéd zajlik a böngésző és a szerver között: a szerver egy véletlenszerű feladványt ad, a böngésző a jelszóból származó titkos kulccsal megoldja, és a szerver ellenőrzi a választ. A jelszó közben egyetlen pillanatra sem hagyja el a böngészőt.

Ha a bejelentkezés sikertelen (pl. hibás jelszó), a rendszer hibaüzenetet jelenít meg. Öt sikertelen kísérlet után a fiók 15 percre zárolódik, hogy a brute-force támadásoknak ne legyen esélye.



3.2. ábra. A bejelentkezési felület — a felhasználó a megszokott módon adja meg nevét és jelszavát

3.5. Kétfaktoros azonosítás (TOTP) beállítása és használata

A felhasználók opcionálisan bekapcsolhatják a kétfaktoros azonosítást (2FA), amely egy második védelmi réteget biztosít a bejelentkezéshez.

Beállítás:

1. A csevegőfelületen a felhasználó megnyitja a beállítások menüt.
2. A „Kétfaktoros azonosítás bekapcsolása” opcióra kattintva megjelenik egy **QR-kód**.

3. A felhasználó egy autentikátor alkalmazással (pl. Google Authenticator, Authy) beolvassa a QR-kódot.
4. Megerősítésként meg kell adnia az alkalmazás által generált első 6 jegyű kódot.

Használat bejelentkezésakor: Ha a 2FA aktív, a felhasználónevét és jelszavát követően egy további lépés jelenik meg: a rendszer bekéri az autentikátor alkalmazás által generált aktuális 6 jegyű kódot. Csak a helyes kód megadása után engedélyezi a belépést.

Kikapcsolás: A 2FA a beállítások menüben bármikor kikapcsolható. Biztonsági okokból ehhez meg kell adni az érvényes 6 jegyű TOTP kódot.

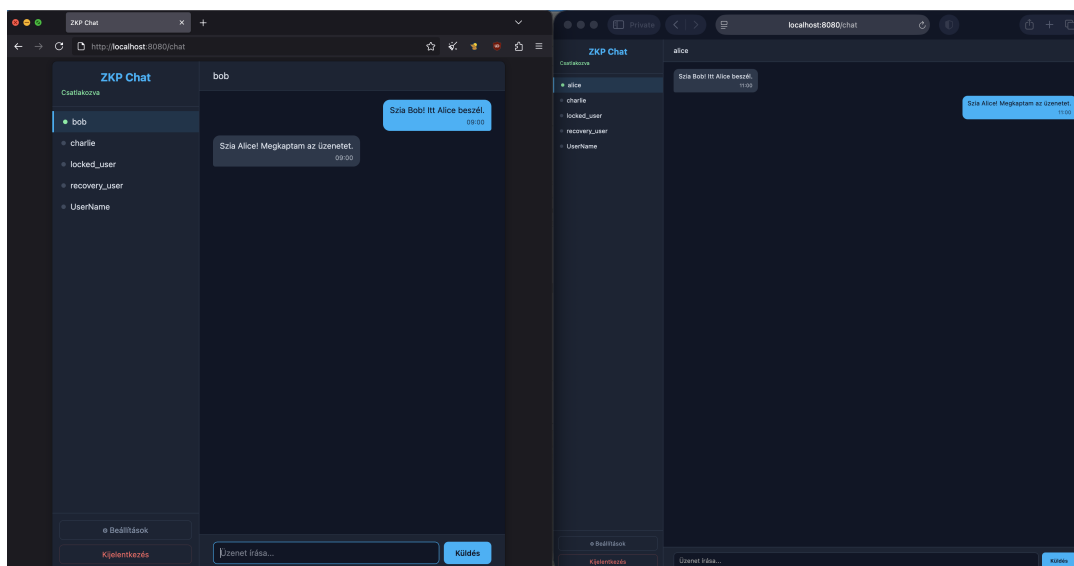
3.6. Csevegés — valós idejű üzenetküldés

A sikeres bejelentkezés után a felhasználó a csevegőfelületre kerül, amely a következő elemekből áll:

- **Felhasználólista (oldalsáv):** Az összes regisztrált felhasználó megjelenik, mellettük jelzéssel, hogy éppen online (zöld) vagy offline (szürke) állapotban vannak. Az állapotjelzők valós időben frissülnek.
- **Beszélgetési ablak:** Egy felhasználóra kattintva megjelenik a vele folytatott beszélgetés. A küldött és fogadott üzenetek vizuálisan elkülönülnek (eltérő szín és elhelyezés).
- **Üzenetküldés:** A szövegbeviteli mezőbe írva és az „Enter” billentyűvel vagy a küldés gombbal az üzenet azonnal megjelenik a címzettnél.

Az üzenetek valós időben érkeznek, így nincs szükség az oldal frissítésére. Ha a címzett éppen offline, az üzenetet a rendszer eltárolja, és a következő bejelentkezésakor automatikusan kézbesíti. A korábbi üzenetek visszagörgethető előzmény formájában is elérhetők.

Ha a hálózati kapcsolat megszakad, a rendszer automatikusan megpróbál újra-csatlakozni a háttérben.



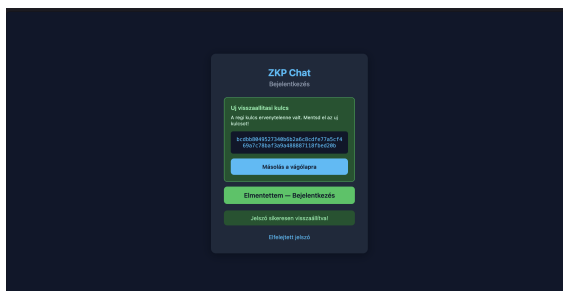
3.3. ábra. A csevegőfelület két felhasználó böngészőjében — a bal oldali panelen a felhasználólista online jelzéssel

3.7. Jelszóvisszaállítás recovery kulccsal

Ha a felhasználó elfelejti a jelszavát, a regisztrációkor kapott recovery kulccsal állíthatja helyre hozzáférését:

1. A bejelentkezési oldalon az „Elfelejtett jelszó” linkre kattint.
2. Megadja a felhasználónevét és a recovery kulcsot.
3. A rendszer a háttérben ugyanazt a kriptográfiai bizonyítási eljárást hajtja végre, mint a normál bejelentkezésnél, ezúttal a recovery kulccsal.
4. Sikeres ellenőrzés után a felhasználó megadhat egy új jelszót.
5. A rendszer generál egy új recovery kulcsot is, amelyet a felhasználónak újra el kell mentenie. A korábbi recovery kulcs érvénytelenné válik.

A jelszóvisszaállítás során is érvényesül a fiókjárolás: öt sikertelen kísérlet után a fiók ideiglenesen nem elérhető.



3.4. ábra. Sikeres jelszóvisszaállítás — a rendszer új recovery kulcsot generál, amelyet a felhasználónak újra el kell mentenie

3.8. Hibaüzenetek és teendők

A rendszer használata során a felhasználó az alábbi hibaüzenetekkel találkozhat:

- **„Hibás felhasználónév vagy jelszó”** — A megadott jelszó nem egyezik a regisztrálttal. Érdemes ellenőrizni a begépelte jelszót, különös figyelmet fordítva a kis- és nagybetűkre.
- **„A fiók ideiglenesen zárolva”** — Öt egymást követő sikertelen bejelentkezési kísérlet után a fiók 15 percre zárolódik. Meg kell várni a zárolás feloldását, majd újra kell próbálni. Ha a jelszó nem jut a felhasználó eszébe, a jelszóvisszaállítás funkció használható.
- **„Érvénytelen TOTP kód”** — A megadott 6 jegyű kód helytelen vagy lejárt. Az autentikátor alkalmazás által generált kódok 30 másodpercenként változnak, ezért a legfrissebb kódot kell megadni.
- **„Érvénytelen recovery kulcs”** — A jelszóvisszaállításhoz megadott recovery kulcs nem egyezik a tárolttal. Fontos meggyőződni arról, hogy a legutóbb kapott recovery kulcsot használja a felhasználó (a jelszóvisszaállítás után mindig új kulcs generálódik).
- **„A kapcsolat megszakadt”** — A hálózati kapcsolat elveszett a szerver felé. A rendszer automatikusan megpróbál újracsatlakozni. Ha a probléma tartósan fennáll, ellenőrizni kell az internetkapcsolatot és a szerver elérhetőségét.
- **„A munkamenet lejárt”** — A bejelentkezési token érvényessége lejárt, és az automatikus frissítés nem sikerült. A rendszer átirányít a bejelentkezési oldalra, ahol újra be kell jelentkezni.

II. rész

Fejlesztői dokumentáció

4. fejezet

Elméleti alapok

A Schnorr-protokoll biztonsági garanciái algebrai struktúrákon és egy számítás-elméleti nehézségi feltételezésen nyugszanak. A fejezet a véges testektől indul, eljut a ciklikus csoportokon át a diszkrét logaritmus problémáig, majd a szakirodalmi áttekintéssel zárul, amely a dolgozatban alkalmazott megoldást kontextusba helyezi.

4.1. Véges testek és moduláris aritmetika

A modern aszimmetrikus kriptográfiai primitívek jelentős része véges matematikai struktúrák felett operál. Kiindulópontként tekintsük az n modulusú egész maradékosztályok halmazát: $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$, amelyen az összeadás és a szorzás modulo n értendő [4]. Az összeadás mindig elvégezhető, így $(\mathbb{Z}_n, +)$ minden n -re Abel-csoport. A szorzásra nézve azonban $\mathbb{Z}_n$ általában nem test: egy $a \in \mathbb{Z}_n$ elemnek pontosan akkor létezik multiplikatív inverze, ha $\gcd(a, n) = 1$.

Kriptográfiai szempontból kiemelkedő szerepe van a prím modulusú esetnek. Ha p prím, akkor $\mathbb{Z}_p$ minden nullától különböző eleme invertálható, ezért $(\mathbb{Z}_p, +, \cdot)$ véges test, amelyet $\mathbb{F}_p$ -vel is jelölünk [5]. A testtulajdonság (azaz minden nullától különböző elem szorzásra nézve is invertálható) pontosan prím modulusnál áll fenn. A továbbiakban p mindig prímszámot jelöl.

A $\mathbb{F}_p$ test nem nulla elemeiből álló multiplikatív csoport, amelyet $\mathbb{F}_p^* = \mathbb{Z}_p^*$ -gal jelölök, formálisan:

$$\mathbb{Z}_p^* = \{a \in \mathbb{Z}_p \mid \gcd(a, p) = 1\} = \{1, 2, \dots, p-1\}. \quad (4.1)$$

Ezen csoport rendje $|\mathbb{Z}_p^*| = \varphi(p) = p-1$, ahol φ az Euler-féle totient-függvény [5].

4.2. Ciklikus csoportok és generátorelemek

A hitelesítési protokoll paraméterei egy megfelelő tulajdonságokkal rendelkező, véges ciklikus csoport megválasztását követelik meg. A Schnorr-protokollban a $\mathbb{Z}_p^*$ csoport egy q prím rendű részcsoporthát, G -t alkalmazzuk, ahol q valódi osztója a $(p-1)$ -nek ($q \mid p-1$, $q < p-1$). A $\mathbb{Z}_p^*$ csoport rendje $|\mathbb{Z}_p^*| = p-1$, ami ($p=3$ kivételével) nem prímszám; ezért a protokoll nem a teljes $\mathbb{Z}_p^*$ -ot, hanem annak egy q prím rendű G részcsoporthát használja.

A ciklikus csoport definíciójából adódóan létezik benne legalább egy $g \in G$ elem, az úgynevezett generátor, amelynek ismételt csoportműveletbe vonásával (multiplikatív jelölés esetén hatványozásával) a csoport minden eleme előállítható. A generált részcsoporthat a következőképpen írható fel [4]:

$$G = \langle g \rangle = \{g^k \mid k \in \mathbb{Z}_q\}. \quad (4.2)$$

Az elem g rendje pontosan q (azaz $\text{ord}(g) = q$), és a részcsoporthat elemszáma $|G| = q$ [5].

4.3. A Diszkrét Logaritmus Probléma (DLP) és annak aszimmetriája

A protokollok elméleti biztonsága olyan egyirányú (trapdoor nélküli) aszimmetriát mutató matematikai problémákon alapul, amelyeknél az „előrefelé” történő számítás hatékony (polinom idejű), míg az inverz művelet kiszámítása jelenlegi ismereteink szerint exponenciális komplexitású. A Schnorr-protokoll esetében ez az alapozó nehéz probléma a Diszkrét Logaritmus Probléma (Discrete Logarithm Problem, DLP).

Legyen G egy q prím rendű ciklikus csoport, és legyen $g \in G$ a csoport egy generátoreleme. A DLP a következőképpen definiálható [6]: Adott G , a generátor g , valamint egy véletlenszerűen választott csoportelem $h \in G$. Keressük azt az egyedi $x \in \mathbb{Z}_q$ kitevőt (amely a protokollban a titkos kulcs szerepét tölti be), amelyre teljesül a következő egyenlőség:

$$h = g^x. \quad (4.3)$$

A moduláris hatványozás, azaz a nyilvános h kulcs kiszámítása az x titkos kulcs és a g generátor ismeretében hatékonyan elvégezhető például a gyors hatványozás

(square-and-multiply) algoritmusával [7].

Ezzel szemben, ha a csoport paramétereit megfelelően nagy méretűre választom (kellően nagy p prímszám esetén), az inverz művelet, vagyis az $x = \log_g(h)$ érték meghatározása számításelméletileg nehéz. Nem ismert olyan klasszikus számítógépen futtatható algoritmus, amely polinom idő alatt képes lenne megoldani a problémát egy általános, kellően nagy prím rendű csoport esetén [6]. Ez a számítási aszimmetria biztosítja a Schnorr-protokoll zero-knowledge tulajdonságát és hamisíthatatlanságát: a bizonyító (Prover) használhatja az x titkos kulcsot, miközben a hitelesítő (Verifier) vagy egy passzív megfigyelő a publikusan elérhető h és g elemekből nem képes azt ésszerű időn belül visszanyerni.

4.4. A Schnorr-protokoll

A Schnorr-féle azonosítási protokoll [2] a fentiekben bevezetett algebrai struktúrákra és a diszkrét logaritmus probléma számítási aszimmetriájára épül.

4.4.1. A protokoll áttekintése

A Schnorr-protokoll egy hárommenetes, interaktív, zéró tudású azonosítási séma, amely a diszkrét logaritmus problémán alapul. A protokollban két szereplő vesz részt:

- **Bizonyító ($\mathcal{P}$):** A felhasználó, aki a titkos kulcs (x) birtokában bizonyítja személyazonosságát.
- **Hitelesítő ($\mathcal{V}$):** A szerver, amely a nyilvános kulcs (y) felhasználásával ellenőrzi a bizonyítást.

4.4.2. Rendszerparaméterek

A rendszer paraméterei nyilvánosan elérhetők és a következők:

- Egy nagy p prímszám (a modulus).
- Egy q prím, amely osztója a $(p - 1)$ -nek.
- Egy $g \in \mathbb{Z}_p^*$ elem, amelynek rendje pontosan q (azaz g a G részcsoport generátora, nem a teljes $\mathbb{Z}_p^*$ -é).

A felhasználó titkos kulcsa egy véletlenszerűen választott $x \in \{1, 2, \dots, q-1\}$ egész szám. A hozzá tartozó nyilvános kulcs:

$$y = g^x \pmod{p}. \quad (4.4)$$

4.4.3. A protokoll lépései

A három menet a következőképpen írható le:

1. Elköteleződés (Commitment): A bizonyító választ egy véletlen $k \in \{1, 2, \dots, q-1\}$ értéket és kiszámítja:

$$t = g^k \pmod{p}. \quad (4.5)$$

A t értéket elküldi a hitelesítőnek.

2. Kihívás (Challenge): A hitelesítő generál egy véletlenszerű $c \in \{0, 1, \dots, 2^l-1\}$ kihívás értéket (ahol l a biztonsági paraméter), és elküldi a bizonyítónak. A kihívás értékét a szerver rövid ideig rögzíti egy memóriabeli tárolóban (**ChallengeStore**), lejáratási idővel (TTL) ellátva.

3. Válasz (Response): A bizonyító kiszámítja:

$$s = (k - c \cdot x) \pmod{q} \quad (4.6)$$

és elküldi az s értéket a hitelesítőnek.

4.4.4. Ellenőrzés (Verification)

A hitelesítő a következő egyenlőség teljesülését vizsgálja:

$$g^s \cdot y^c \equiv t \pmod{p}. \quad (4.7)$$

Ez azért helyes, mert:

$$g^s \cdot y^c = g^{k-cx} \cdot g^{xc} = g^k = t. \quad (4.8)$$

Sikeres ellenőrzés esetén a hitelesítő elfogadja a bizonyítást, és a rendszer egy JWT tokenet [8] állít ki a kliensnek.

4.4.5. Biztonsági tulajdonságok

A Schnorr-protokoll a következő biztonsági garanciákat nyújtja [2, 1]:

- **Teljesség:** Amennyiben a bizonyító ismeri a helyes titkos kulcsot, a hitelesítő mindig elfogadja a bizonyítást.
- **Megbízhatóság:** Egy x titkos kulcsot nem ismerő támadó csak elhanyagolhatóan kis valószínűséggel képes érvényes választ generálni.
- **Zéró tudás:** Az interakció során a hitelesítő semmilyen, az x titkos kulcsra vonatkozó információt nem szerez meg, hiszen a protokoll szimulálható a titkos kulcs ismerete nélkül.

Az implementáció során a rendszer kiegészítő biztonsági intézkedéseket is alkalmaz: kérészám-korlátozást (rate limiting) és fiókjárolást az ismételt sikertelen bejelentkezési kísérletek ellen.

4.5. Szakirodalmi áttekintés — ZKP-alapú hitelesítés

A ZKP-alapú hitelesítés nem új kutatási terület: az elmúlt évtizedben számos megközelítés született, a gráf-izomorfizmustól a zk-SNARK-okon át a rácsalapú konstrukciókig. Az alábbi áttekintés kontextusba helyezi a dolgozatban alkalmazott Schnorr-protokollt, és bemutatja a terület jelenlegi állapotát meghatározó alternatív megoldásokat.

4.5.1. Hagyományos jelszó-alapú hitelesítés sebezhetőségei

A jelszó-alapú hitelesítés legsúlyosabb támadási formája az offline szótártámadás: a támadó passzívan lehallgatja a hitelesítési forgalmat, majd a megszerzett ellenőrző értékekre (verifier) szótárból származó jelszó-jelölteket próbál ki. A PAKE-protokollokkal szemben alapelvárás, hogy az egyetlen megvalósítható támadás az *online* kísérletezés legyen, amelynek sikere felülről becsülhető a próbálkozások számával és a szótár méretével [9]. Ennek formális kerete Toorani dolgozatában a BPR-modell, amely megkülönböztethetőségi játékként definiálja a PAKE-biztonságot: a

támadó előnye a valódi és a véletlen session-kulcs megkülönböztetésében nem lehet nagyobb az online találgatási valószínűségnél [9].

A zéró tudású bizonyítások ebben a modellben azért kínálnak előrelépést, mert a bizonyító fél *sosem* osztja meg titkos adatát az ellenőrzővel: a ZKP konstrukcióiból formálisan igazolható, hogy az interakció során a titkos kulcsra vonatkozó információ nem szivárog ki, ami közvetlenül csökkenti a szerver-oldali kompromittálás és adatszivárgás hatásait [10].

4.5.2. ZKP hitelesítési architektúrák az irodalomban

Az elmúlt évtizedben több kutatócsoport kísérelt meg ZKP-alapú hitelesítési rendszereket konkrét kontextusokhoz (IoT, webes portálok, mobil 2FA) szabni. A bibliometriai áttekintések közül Pathak és szerzőtársai munkája ad jó képet a terület szerkezetéről: kiemelik többek között a J-PAKE optimalizált változatait (RO-J-PAKE, CRS-J-PAKE), amelyek két zéró tudású bizonyítással kevesebbet használnak az eredetinél, valamint a SeDiCi 2.0-t, amely kifejezetten mobil és felhőalapú környezetekhez kínál kétfaktoros ZKP-hitelesítést [11].

Az architektúra-szintű megközelítések közös vázát Chen és szerzőtársai az IoT-alkalmazások áttekintésében foglalták össze. A tipikus rendszer három fázisból épül fel: az inicializálás előállítja a nyilvános kulcsokat, a regisztráció tárolja a hashelt jelszót vagy a nyilvános kulcsot, a hitelesítés pedig kihívás-válasz alapú. A Fiat–Shamir heurisztika ebben a modellben kulcsszerepet kap, mivel az interaktív bizonyítások nem-interaktívvá alakításával csökkenti a hálózati kommunikációt, ami különösen fontos tulajdonság nagy késleltetésű IoT-környezetben [12]. Egy ilyen konkrét IoT-konstrukciót mutatnak be Dwivedi és szerzőtársai: náluk a protokoll egy algoritmus-hármas (KeyGen, Prover, Verifier) köré épül, a regisztráció pedig PAKE-alapú eljárással számolja ki a szerverhez érkező nyilvános paramétert, úgy hogy a jelszó egyszerű szöveggént sosem kerül a szerverre [13].

A gyakorlati teljesítmény kérdéseire a zk-SNARK-alapú captive portál-hitelesítés Thiagarajan rendszere ad mérési referenciát: a szerver kihívást küld, a kliens pedig lokálisan tárolt titkos kulccsal bizonyítást számol. 1000 párhuzamos felhasználónál az átlagos ellenőrzési idő 0,18 másodperc, a sikerességi arány 97,5 % volt [14]. Ez nagyságrendileg lassabb, mint a Schnorr-protokoll szerver-oldali két-három moduláris hatványozása, ám valós rendszerben is bőven alkalmazható, cserébe kényelmesebb

interakciós modellt (egyetlen kör) ad.

Más matematikai alapokra építő megoldás Saluja és Bhatia gráf-izomorfizmuson alapuló sémája, amelyben minden kriptográfiai számítás (titkos kulcs, gráfok konstrukciója) a böngészőben zajlik: a bizonyító egy véletlenszerűen permutált gráfot küld, a szerver bitenkénti kihívásokat ad, és csak az izomorfizmus ismeretében adható helyes válasz. Tíz iteráció után a csalás valószínűsége 2^{-10} alá kerül [15]. Asimi és szerzőtársai SAVP rendszere ehhez képest más úton jár: a statikus jelszót nem kiváltja, hanem *virtuális jelszóvá* alakítja egy dinamikus rotációs algoritmussal, amely a jelszót és egy felhasználóhoz kötött véletlen sőt kiszámíthatatlan származtatott értékkel kombinál, a kölcsönös hitelesítés pedig nonce-alapú kihívás-cserével történik [16].

4.5.3. PAKE protokollcsalád

A jelszó-alapú hitelesített kulcscsere (Password-Authenticated Key Exchange, PAKE) protokollcsalád szorosan kapcsolódik a ZKP-alapú hitelesítéshez, mivel számos PAKE-variáns zéró tudású bizonyítási primitívekre épül. Ennek egyik legszebb példája a J-PAKE (*Password-Authenticated Key Exchange by Juggling*), amely a Schnorr-féle nem-interaktív zéró tudású bizonyítást (NIZK) használja építőelemként. Biztonságát a Döntési Diffie–Hellman (DDH) feltételezésre vezeti vissza, és két körben valósítja meg a kulcscserét: a felek efemer nyilvános kulcsokat és a hozzájuk tartozó NIZK-eket cserélnek, amelyeket a közös session-kulcs származtatása előtt mindkét félnek ellenőriznie kell [17]. A dolgozatban választott Schnorr-protokoll ugyanabba a „családba” illeszkedik, csak más alkalmazási céllal (hitelesítés, nem kulcscsere).

A PAKE-variánsok széles összehasonlítására jó kiindulópont Vazquez Sandoval és szerzőtársai PakeMail munkája, amely magát az SPAKE2 protokollt használja — a felek „vakított” (blinded) Diffie–Hellman tagokat cserélnek, majd MAC-alapú kulcsmegerősítést (key confirmation) végeznek —, a dolgozatuk második felében pedig hét PAKE-protokollt (SPAKE2, PFS-SPAKE2, OPAQUE, J-PAKE, KV-SPOKE, RLWE-PAK, RLWE-PPK) vetnek össze a körök száma, a kulcs-megerősítés támogatása, a forward secrecy és a nehézségi feltételezés szerint, beleértve rácsalapú (lattice-based), poszt-kvantum kompatibilis jelölteket is [18].

A klasszikus megközelítést Kobara és Imai Pretty-Simple PAKE protokollja kép-

viseli: a DDH-nehézségre épített titkosító amplifikációval (secrecy amplification) a véletlen kitevők elfedik a jelszót, és így a lehallgatott forgalomból nem vezethető vissza. A kölcsönös hitelesítést kulcsolt hash-függvénnyel és külön tagekkel végzik a kliens és a szerver felé [19]. Egy újabb, másik problémára optimalizált változat Fomichev és szerzőtársai FastZIP rendszere, amelyben a fPAKE (*fuzzy PAKE*) az EKE-re (Encrypted Key Exchange) épül: alacsony entrópiájú titkokból (tipikusan 40–60 bites bemenetből) párhuzamos PAKE-példányokkal állít elő magas entrópiájú kulcsot. A szerzők elemzése szerint aktív támadó munkamenetenként legfeljebb egyetlen tippelésre korlátozódik, a 90 % fölötti hasonlósági küszöb esetén pedig az offline támadás valószínűsége 2^{-20} alá esik [20].

4.5.4. ZKP-alapú jelszó-visszaállítás és kulcskezelés

Egy ZKP-alapú hitelesítési rendszer sajátos kihívása a jelszóvisszaállítás, hiszen a szerver definíció szerint nem rendelkezik a felhasználó titkos adataival. Az irodalom több iránya is foglalkozott ezzel a problémával, javarészt az *Oblivious PRF* primitív (OPRF, *feledékeny álvéletlen függvény*) köré építve.

A dolgozatban választott „second verifier” megoldás (recovery-verifier) egy egyszerű, szerver-állapotot nem igénylő változat; a szakirodalom ennél finomabb konstrukciókat is ismer. Little és szerzőtársai egy részlegesen feledékeny PRF-re épülő fiók-helyreállítási protokollt mutatnak be, amelyben additív homomorf titkosítás gondoskodik arról, hogy sem a kliens, sem a szerver ne lássa a PRF köztes állapotait; a szótártámadások ellen memória-igényes, lassú hash-függvény véd [21]. Ezt általánosítja a jelszóvédett kulcs-visszakérés (*Password-Protected Key Retrieval*, PPKR) problémája, amelyet Faller és szerzőtársai vizsgáltak: a cél, hogy a felhasználó kizárólag a jelszavával tudjon magas entrópiájú kulcsot letölteni a szerverről, miközben a szerver sem a kulcshoz, sem a jelszóhoz nem fér hozzá. OPRF-alapú protokolljuk Lev-3 biztonsági szintje még teljesen kompromittált hardveres biztonsági modul (HSM) esetén is védett marad, mert a támadónak felhasználónként egyedi offline támadást kellene végrehajtania [22].

A szerver-oldali jelszó-házirend ZKP-keretben nehezebb kérdés, mert a szerver nem látja a jelszót, de mégis érvényesítenie kell a szabályokat. Zhang és szerzőtársai B-OPRF (*Blocklisted Oblivious PRF*) keretrendszere erre ad választ: az augmented PAKE (aPAKE) protokollokba ágyazva tiltólistán szűri a gyakori jelszavakat úgy,

hogy közben nem ismeri meg a felhasználó jelszavát (a regisztrációs fázis 1500 elemű klaszter-tiltólistával 1,42 másodperc alatt fut le) [23]. Poszt-kvantum irányban Nguyen és szerzőtársai zero-knowledge password policy check (ZKPPC) konstrukciója a Short Integer Solution (SIS) rácsproblémát és Stern-típusú protokollokat használ: 52 párhuzamos iterációval éri el a megkívánt zéró tudás biztonsági szintet, anélkül hogy a jelszó vagy annak szerkezete bármikor felfedésre kerülne [24].

4.5.5. A Schnorr-protokoll választásának indoklása

A fenti áttekintés jól mutatja, hogy a ZKP-alapú hitelesítés területén bőséges a választék. A dolgozat kontextusában mégis a Schnorr-protokoll mellett döntöttem, és az alábbiakban ezt a döntést indokolom.

Van Laer és szerzőtársai egy olyan kliens-szerver hitelesítési rendszert terveztek, amelyben a Schnorr-féle azonosítási sémát a Fiat-Shamir heurisztikával nem-interaktív formára alakították, és memóriaigényes (memory-hard) hash függvénnyel (scrypt) kombinálták a szótártámadások elleni védelemhez. A moduláris felépítés (JavaScript kliens könyvtár és Java szerver könyvtár, JSON-alapú bizonyítás-objektumok cseréjével) jól demonstrálja a Schnorr-protokoll webes környezetbe való integrálhatóságát [25].

Major és szerzőtársai egy káosz-alapú rendszerbe ágyazva alkalmazták a Schnorr-féle nem-interaktív zéró tudású bizonyítást, 2048 bites DSA csoportparaméterekkel. A szerver-oldali ellenőrzés két moduláris hatványozást és egy kriptográfiai hash függvény kiértékelését igényli, ami kellően hatékony a valós idejű hitelesítéshez [26].

Érdemes megemlíteni az alternatív ZKP-megközelítéseket is: Powsic kvadratikus reziduumokon alapuló protokollt tervezett, amelyet EAP (Extensible Authentication Protocol) módszerként specifikált. A szerző kiemeli, hogy a kulcsnyújtás (key-stretching) ZKP-rendszereknél sajátos kihívást jelent, mivel a tárolt nyilvános érték közvetlenül szükséges az ellenőrzéshez [27]. Ez az észrevétel rávilágít a Schnorr-protokoll előnyére: a kliens-oldali PBKDF2 kulcsszármaztatás természetesen illeszkedik a protokollba, mivel a szerver kizárólag a származtatott nyilvános kulcsot ($y = g^x \bmod p$) tárolja.

A Schnorr-protokoll mellett több gyakorlati szempont is szólt. A legfontosabb, hogy a hárommenetes interakció (commitment, challenge, response) közvetlenül leképezhető három HTTP kérés-válasz párra: ez a zk-SNARK-alapú vagy gráf-

izomorfizmus-alapú rendszereknél jóval egyszerűbb REST API végpont-struktúrát eredményez. A kliens oldalán sincs szükség speciális hardverre, mivel a szükséges kriptográfiai műveletek (a moduláris hatványozás és a hash-számítás) a modern böngészők beépített BigInt és Web Crypto API-jaival elvégezhetők. A protokoll klasszikus biztonsága ráadásul a jól tanulmányozott diszkrét logaritmus probléma nehézségére vezethető vissza [2, 6], és, ahogy a dolgozatban is látható lesz, természetesen kombinálható PBKDF2 kulcsszármaztatással és TOTP kétfaktoros azonosítással.

5. fejezet

Használt technológiák

A rendszer fejlesztése során több technológiai döntést kellett meghoznom — a szervernyelvtől az adatbázis-megoldáson át a kliens-oldali kriptográfiai eszközökig. A fejezet az egyes rétegeket külön alfejezetekben veszi végig, mindig a választás indokával együtt.

5.1. Go 1.24

A szerver-oldali logika a Go programozási nyelv [28] 1.24-es verziójával készült. A nyelv erős típusrendszere, beépített párhuzamossági primitívjei (goroutine-ok, channel-ek) és átfogó standard könyvtára különösen jól illik a WebSocket-alapú valós idejű rendszerhez: a goroutine-ok minden WebSocket kapcsolatot külön könnyűsúlyú szálon kezelnek, így több ezer egyidejű kapcsolat is hatékonyan kiszolgálható. A `crypto/rand` és `math/big` csomagokkal a Schnorr-protokoll kriptográfiai műveletei külső függőség nélkül megvalósíthatók, a fordító egyetlen statikusan linkelt binárist állít elő, a beépített `testing` csomag pedig külön keretrendszer nélkül támogatja mind az egység-, mind az integrációs teszteket.

5.1.1. gorilla/websocket

A gorilla/websocket [29] egy széles körben használt, jól tesztelt WebSocket implementáció Go nyelven. A könyvtár teljes körűen támogatja a WebSocket protokollt [30]: a bináris és szöveges üzenetek mellett a ping/pong kereteket és a kapcsolatbontási kódokat is. A rendszerben a WebSocket szerver biztosítja a kétirányú valós idejű üzenetküldést a kliens és a szerver között, a jelenlét-állapotok (presence) broadcast-

alapú szórását az aktív kliensek felé, valamint a 30 másodpercenkénti ping-et, amely a kapcsolatot életben tartja.

5.1.2. chi/v5 HTTP router

A chi/v5 [31] egy könnyűsúlyú, idiomatikus HTTP router Go-hoz, amely a standard `net/http` csomagra épül. A népszerűbb Gin vagy Echo alternatívákkal szemben nem hoz magával saját kontextus-kezelést vagy válasz-író, kizárólag a standard könyvtár interfészeire épít: bármely `http.Handler` kompatibilis middleware (hitelesítés, CORS, naplózás) változtatás nélkül beilleszthető a láncba. URL-paramétereket és csoportosított útvonalakat is támogat, a rendszerben ez kezeli az összes REST API végpontot: a ZKP hitelesítési, TOTP, felhasználókezelési és chat-előzmény kéréseket.

5.1.3. SQLite (modernc.org/sqlite, tiszta Go)

Az adatok tartós tárolásához a rendszer SQLite relációs adatbázist [32] használ, a `modernc.org/sqlite` [33] tiszta Go implementáción keresztül. A tiszta Go implementáció CGo-mentes fordítást tesz lehetővé, ami egyszerűbb keresztfordítást (cross-compilation) és telepítést eredményez. Az adatbázis egyetlen fájlként (`zqp-chat.db`) a szerver mellett él, külön adatbázis-szerver nélkül, ami illeszkedik a rendszer „külső függőség nélküli” tervezési elvéhez. A tesztkörnyezet ugyanezt a könyvtárat in-memory módban (`:memory:`) használja, így a tesztek gyorsak és izoláltak maradnak.

5.1.4. JWT (HMAC-SHA256)

A session kezelés JSON Web Tokeneken [8] alapul, HMAC-SHA256 (HS256) aláírással. A szerver a sikeres hitelesítés után állít ki tokent, amely a felhasználó azonosítóját (`sub claim`) és az érvényességi időt tartalmazza. Az állapotmentes token-modell illeszkedik a REST API architektúrához: a szerver nem tart fenn session-állapotot. A 2 órás érvényességi idő egyensúly a biztonság és a felhasználói kényelem között, a kliens pedig a lejárat előtt automatikusan frissíti a tokent, így a felhasználónak nem kell újra bejelentkeznie.

5.1.5. pquerna/otp (TOTP)

A kétfaktoros azonosítás az RFC 6238 [3] szerinti TOTP (Time-Based One-Time Password) mechanizmust alkalmazza. A `pquerna/otp` [34] Go könyvtár lefedi a teljes szükséges funkcionalitást: a TOTP titkos kulcs generálását, a mobilalkalmazással beolvasható QR-kód előállítását (base64 PNG formátumban), valamint az időalapú kódok 30 másodperces ablakkal történő validálását.

5.2. Kliens-oldali technológiák

A kliens-oldali logika Vanilla JavaScript-ben készült, keretrendszer (React, Vue stb.) nélkül, IIFE (Immediately Invoked Function Expression) modul mintával. A projekt fókusza a kriptográfiai hitelesítés és nem a frontend architektúra, a keretrendszer-mentesség pedig kiváltja a build-lépést és csökkenti a komplexitást.

A kliens-oldali kriptográfia a böngésző natív képességeire támaszkodik: a JavaScript beépített `BigInt` típusa végzi a Schnorr-protokollhoz szükséges moduláris hatványozást nagy egészekkel, a Web Crypto API `subtle.deriveBits` metódusa pedig a felhasználó jelszavából származtatja a titkos kulcsot PBKDF2-SHA256-tal. A valós idejű kapcsolatot a böngésző natív WebSocket API-ja biztosítja, automatikus újracsatlakozási logikával kiegészítve. Az üzenetek megjelenítésekor a `escapeHtml` segédfüggvény előzi meg a cross-site scripting (XSS) támadásokat.

6. fejezet

Komponensek felépítése és kapcsolataik

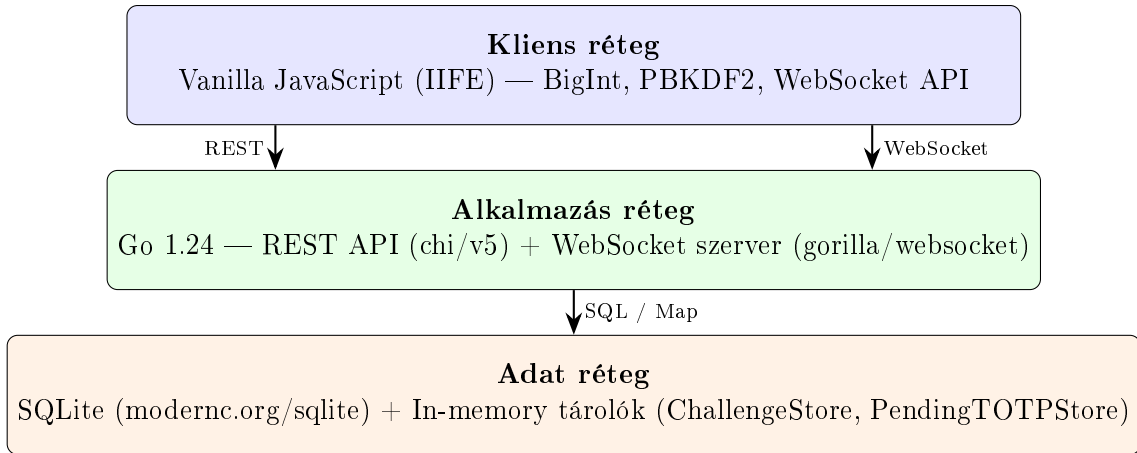
A rendszer tervezésekor az volt a fő szempont, hogy a kriptográfiai hitelesítés és a valós idejű csevegés igényeit egyetlen, önállóan futtatható alkalmazásban egyesítsem. Ebből alakult ki a háromrétegű architektúra, amelynek komponenseit és a tervezési döntések hátterét a fejezet a továbbiakban részletezi.

6.1. Architektúra áttekintés

Az architektúra három rétegre épül:

1. **Kliens réteg:** Vanilla JavaScript alapú webalkalmazás (IIFE modul mintával), amely végrehajtja a kliens-oldali kriptográfiai műveleteket (PBKDF2–SHA256 kulcsszármaztatás, salt és verifier generálás, commitment és response számítás BigInt aritmetikával) és kezeli a felhasználói felületet.
2. **Alkalmazás réteg:** Go 1.24 nyelven írt szerver, amely két fő alrendszert tartalmaz:
 - HTTP REST API (chi/v5 [31] router) — hitelesítés, TOTP, felhasználókezelés, chat előzmények.
 - WebSocket szerver (gorilla/websocket [29]) — valós idejű üzenettovábbítás és jelenlét-kezelés.
3. **Adat réteg:** SQLite [32] relációs adatbázis (modernnc.org/sqlite [33], tiszta Go implementáció, CGo nélkül) a felhasználók és üzenetek tárolásához, vala-

mint memóriabeli (in-memory) tárolók a rövid életciklusú állapotok kezelésére (challenge értékek, TOTP pending tokenek).



6.1. ábra. A rendszer háromrétegű architektúrája

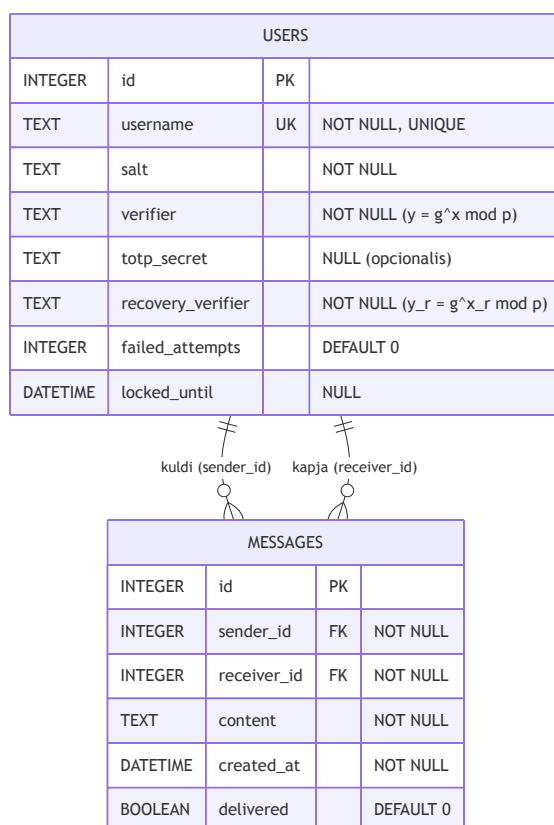
6.2. Adatmodell

A rendszer az alábbi fő adatentitásokat kezeli:

- **Users tábla:** felhasználónév (`username`), salt, verifier ($y = g^x \bmod p$), TOTP titkos kulcs (opcionális), recovery verifier ($y_r = g^{x_r} \bmod p$, a jelszóvisszaállításhoz), sikertelen belépési kísérletek száma (`failed_attempts`), fiókszárolás időpontja (`locked_until`).
- **Messages tábla:** feladó azonosító (`sender_id`), címzett azonosító (`receiver_id`), szöveges tartalom (`content`), időbélyeg (`created_at`), kézbesítési állapot (`delivered`).

A challenge értékek és a TOTP pending tokenek nem az adatbázisban, hanem memóriabeli `sync.RWMutex`-védett Go map struktúrákban tárolódnak, háttérzál segítségével végzett automatikus TTL-alapú lejáratkezeléssel.

Az adatbázis felépítését a 6.2. ábra szemlélteti.



6.2. ábra. Az adatbázis ER diagramja — a Users és Messages táblák kapcsolata

Az alábbi táblázatok részletezik az egyes adatbázis-táblák mezőszerkezetét.

Mező	Típus	Leírás
id	INTEGER (PK)	Automatikusan növekvő elsődleges kulcs
username	TEXT (UNIQUE)	Egyedi felhasználónév
salt	TEXT	32 byte-os véletlenszerű salt (hex)
verifier	TEXT	Nyilvános kulcs: $y = g^x \bmod p$ (hex)
totp_secret	TEXT (NULL)	TOTP titkos kulcs (ha 2FA aktív)
recovery_verifier	TEXT	Recovery nyilvános kulcs: $y_r = g^{x_r} \bmod p$
failed_attempts	INTEGER	Sikertelen belépések száma (alapért.: 0)
locked_until	DATETIME (NULL)	Fiókszárolás lejáratási időpontja

6.1. táblázat. A `users` tábla mezőszerkezete

Mező	Típus	Leírás
id	INTEGER (PK)	Automatikusan növekvő elsődleges kulcs
sender_id	INTEGER (FK)	Feladó felhasználó azonosítója
receiver_id	INTEGER (FK)	Címzett felhasználó azonosítója
content	TEXT	Az üzenet szöveges tartalma
created_at	DATETIME	Az üzenet létrehozásának időbélyege
delivered	BOOLEAN	Kézbesítési állapot (0: várakozik, 1: kézbesítve)

6.2. táblázat. A `messages` tábla mezőszerkezete

6.3. A hibrid kommunikációs réteg (REST + WebSocket)

A rendszer két különálló kommunikációs csatornát alkalmaz: REST API-t a hitelesítéshez, WebSocketet az üzenetváltáshoz. A szétválasztás oka, hogy a két feladat természete eltér: a hitelesítés kérés-válasz alapú és állapotmentes (stateless), míg a csevegés folyamatos, állapottartó (stateful) kapcsolatot igényel. A REST API izolált, jól tesztelhető végpontokat biztosít a Schnorr-protokoll egyes lépéseihez, a WebSocket csatorna pedig kizárólag a már hitelesített felhasználók valós idejű üzeneteiért felel, így az implementáció egyik felét sem terheli a másik feladat állapotkezelése.

6.3.1. A HTTP REST API szerepe

A HTTP REST API kizárólag a rendszer belépési pontjaként funkcionál. Kérés-válasz alapú, állapotmentes protokoll lévén kiválóan illeszkedik a Schnorr-protokoll lépéseinek leképezéséhez. Az autentikációs folyamat a következő HTTP végpontokon keresztül valósul meg:

- `GET /api/auth/params` — a Schnorr-csoport nyilvános paramétereinek (p , q , g) lekérése.
- `POST /api/auth/register` — regisztráció (felhasználónév, salt, verifier, recovery verifier).
- `POST /api/auth/login/init` — a Schnorr-protokoll első fázisa (salt és challenge visszaadása).
- `POST /api/auth/login/verify` — a protokoll záró fázisa (commitment és response ellenőrzése).
- `POST /api/auth/recovery/init` — jelszóvisszaállítás első fázisa.
- `POST /api/auth/recovery/reset` — jelszóvisszaállítás végrehajtása.
- `POST /api/auth/refresh` — JWT token frissítése.

Sikeres kriptográfiai ellenőrzés esetén a REST API egy JWT tokenet állít ki 2 órás érvényességgel, amely a későbbi kommunikáció felhatalmazási eszközeként szolgál.

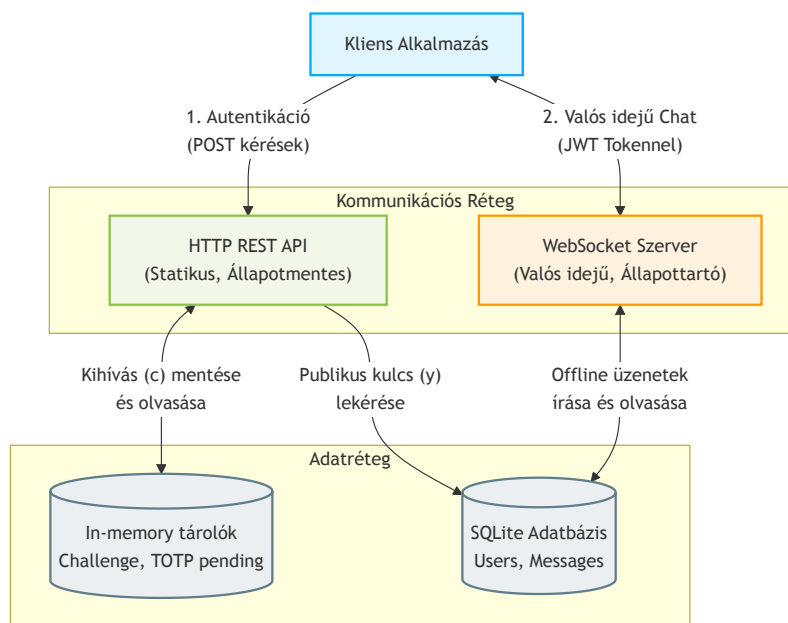
6.3.2. A WebSocket csatorna

A tényleges üzenetváltáshoz a WebSocket protokoll [30] nyújtja a szükséges alacsony késleltetésű, kétirányú kommunikációs csatornát. A folyamat a hitelesítést követően:

1. **Kapcsolatfelvétel:** A kliens WebSocket handshake-et kezdeményez a `/ws?token=<JWT>` végponton, ahol a JWT tokenet query paraméterként továbbítja.
2. **Validáció:** A szerver ellenőrzi a token HMAC-SHA256 aláírását és érvényességi idejét. Sikeres validáció esetén a protokollváltás megtörténik.

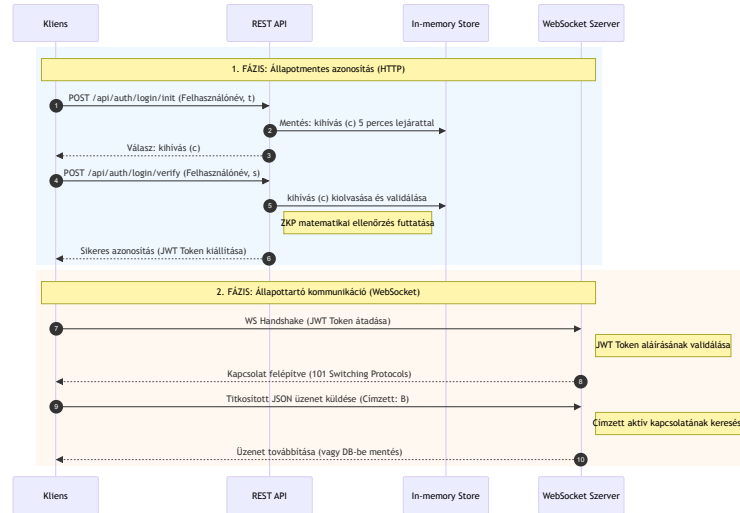
3. **Üzenettovábbítás:** A kliensek közötti üzenetek JSON envelope formátumban utaznak. A szerver (Hub) a beérkező adatsomagot értelmezi, a címzettet azonosítja, és a megfelelő aktív kapcsolatra továbbítja.
4. **Kézbesítés vagy tárolás:** Amennyiben a címzett aktív, a szerver azonnal továbbítja az üzenetet. Inaktív állapot esetén az üzenet az adatbázisba kerül `delivered = 0` állapottal, és a címzett következő csatlakozásakor a rendszer automatikusan kézbesíti.

A rendszer felépítését a 6.3. ábra szemlélteti.



6.3. ábra. A hibrid kommunikációs réteg (REST API és WebSocket) logikai felépítése

A teljes hitelesítési és kommunikációs szekvenciát a 6.4. ábra mutatja be.



6.4. ábra. A Schnorr-autentikáció és a WebSocket kapcsolatfelépítés szekvenciája

6.4. Biztonsági megfontolások

- A szerveren *soha* nem tárolódik jelszó vagy jelszó-hash érték, kizárólag a salt és a verifier (y).
- A challenge értékek memóriabeli tárolóban rövid lejáratú idővel (TTL) rendelkeznek, megakadályozva a replay-támadásokat. Egyszeri felhasználásúak: a sikeres vagy sikertelen ellenőrzés után azonnal törlődnek.
- A JWT tokenek HS256 (HMAC-SHA256) aláírással rendelkeznek, 2 órás érvényességi idővel [8]. A `sub` claim tartalmazza a felhasználónevet. A kliens automatikusan frissíti a tokenet a lejárat előtt 5 perccel.
- Fiókszárolás 5 sikertelen belépési kísérlet után automatikusan aktiválódik (a recovery folyamatra is érvényes).
- A WebSocket kapcsolat hitelesítése a JWT token query paraméterként történő validálásával valósul meg. A szerver 30 másodpercenként ping üzenetet küld (heartbeat), és a lejárt tokenű klienseket automatikusan lecsatlakoztatja.

6.5. Telepítési architektúra

A fejlesztési és futtatási környezet egyszerű és önállóan működik:

1. `go mod tidy` — a függőségek letöltése.

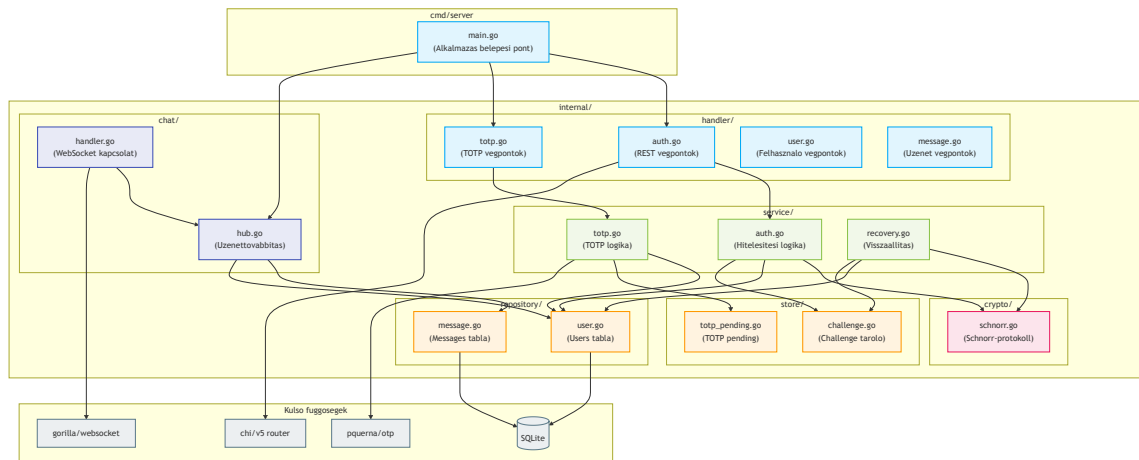
2. `make run` (vagy `go run ./cmd/server`) — a szerver indítása a 8080-as porton.

Az első indításkor a szerver generálja a 2048 bites Schnorr csoport paramétereit (ez akár egy percig is tarthat), amelyeket a `params.json` fájlba ment. A `zpk-chat.db` SQLite adatbázis szintén automatikusan létrejön.

A rendszer a fenti módon külső szolgáltatás-függőség (Redis, üzenetközvetítő stb.) nélkül működik, mivel a `modernc.org/sqlite` könyvtár tiszta Go-ban implementálja az SQLite motort, és a challenge/TOTP állapotok in-memory tárolókban élnek. Egy alternatív, konténerizált telepítési módot a 7.5. alfejezet mutat be.

6.6. Modulstruktúra

A Go szerver belső felépítése jól elkülönített csomagokra (package) tagolódik, amelyek a felelősségi körök mentén szerveződnek. A 6.5. ábra a csomagok közötti függőségi kapcsolatokat mutatja be.



6.5. ábra. A Go szerver csomagstruktúrája és a modulok közötti függőségek

A főbb csomagok és felelősségeik:

- **cmd/server:** Az alkalmazás belépési pontja (a HTTP szerver, a router és a függőségek inicializálása).
- **internal/handler:** A REST API végpontok kezelői (controller réteg), amelyek a HTTP kéréseket fogadják és a service rétegnek delegálják.
- **internal/service:** Az üzleti logika (a Schnorr-hitelesítés, TOTP kezelés és jelszóvisszaállítás koordinálása).

- `internal/crypto`: A Schnorr-protokoll matematikai műveletei (csoportparaméter-generálás, bizonyítás-ellenőrzés).
- `internal/chat`: A WebSocket Hub és a valós idejű üzenettovábbítás logikája.
- `internal/repository`: Az adathozzáférési réteg (Data Access Layer) — SQL lekérdezések a Users és Messages táblákhoz, valamint a memóriabeli tárolók (ChallengeStore, PendingTOTPStore) TTL-alapú lejáratkezeléssel.
- `internal/middleware`: A JWT autentikációs middleware, vagyis a védett végpontok hitelesítési rétege.
- `internal/model`: Az adatmodell struktúrák (User, Message) definíciói.
- `internal/config`: A környezeti változókból olvasott konfigurációs beállítások (port, adatbázis-elérés, JWT secret, paraméter-fájl útvonal).

6.7. Rendszerdiagramok

A fenti ábrák a rendszer két központi aspektusát szemléltetik. A 6.3. ábra a hibrid kommunikációs réteg logikai felépítését mutatja: a REST API és a WebSocket szerver közötti feladatmegosztást, a JWT-alapú felhatalmazási láncot, valamint az adat réteg (SQLite + in-memory tárolók) szerepét. A 6.4. ábra a teljes hitelesítési és WebSocket kapcsolatfelépítési szekvenciát ábrázolja időrendi sorrendben: a Schnorr-protokoll hárommenetes párbeszédétől a JWT kiállításán át a WebSocket csatorna megnyitásáig.

7. fejezet

Az egyes komponensek működése

7.1. ZKP regisztráció és bejelentkezés

A rendszer biztonsági architektúrájának központi eleme a Schnorr-protokollon alapuló jelszómentes hitelesítés, amelynek megvalósítása a kliens-oldali kulcsszármaztatástól a szerver-oldali bizonyítás-ellenőrzésig terjed. Az előző fejezetben bemutatott komponensek szintjén haladva az alfejezetek ezt a teljes láncot kibontják.

7.1.1. Regisztráció (US-01)

A regisztrációs folyamat során a kliens-oldali JavaScript kód (`schnorr.js` modul) generálja a kriptográfiai paramétereket:

1. A kliens véletlenszerűen választ egy 32 byte-os salt értéket.
2. PBKDF2-SHA256 kulcsszármaztatással kiszámítja a titkos kulcsot: $x = \text{KDF}(\text{jelszó}, \text{salt})$.
3. Kiszámítja a verifier-t (nyilvános kulcsot): $y = g^x \bmod p$.
4. A kliens a felhasználónevet, a salt-ot és a verifier-t (y) a `POST /api/auth/register` végpontra küldi.
5. A Go szerver a kapott adatokat az SQLite adatbázis `users` táblájában tárolja.

A szerveren semmilyen titkos adat — sem jelszó, sem hash — nem tárolódik. A jelszó soha nem hagyja el a klienst.

7.1.2. Bejelentkezés — Challenge (US-02)

A Schnorr-protokoll első menete:

1. A kliens elküldi a felhasználónevet a `POST /api/auth/login/init` végpontra.
2. A szerver visszaküldi a tárolt salt értéket, így a kliens a PBKDF2-SHA256 függvénnyel újra kiszámíthatja az x titkos kulcsot.
3. A szerver kriptográfiailag biztonságos véletlenszám-generátorral előállít egy c kihívás értéket.
4. A kihívást memóriabeli tárolóban (`ChallengeStore`, `sync.RWMutex`-védett Go map) rögzíti, 5 perces TTL-lel.
5. A c értéket visszaküldi a kliensnek.

7.1.3. Bejelentkezés — Proof Verify (US-03)

A protokoll záró menete, amelynek kliens-oldali JavaScript implementációja (`schnorr.js`) az alábbi:

```

1 function createProof(x, challengeHex) {
2     const c = BigInt('0x' + challengeHex);
3     const k = generateRandomScalar();
4     const t = modPow(params.g, k, params.p); // commitment
5     const s = mod(k - c * x, params.q);      // response
6     return {
7         commitment: t.toString(16),
8         response:    s.toString(16),
9     };
10 }
```

7.1. forráskód. A Schnorr-bizonyítás kliens-oldali generálása (`schnorr.js`)

A `modPow` függvény a JavaScript beépített `BigInt` típusával végzi a moduláris hatványozást. A `generateRandomScalar` kriptográfiailag biztonságos véletlenszámot állít elő a `crypto.getRandomValues` API segítségével. A lépések részletesen:

1. A kliens generál egy véletlen k értéket és kiszámítja a commitment-et: $t = g^k \bmod p$.

2. Kiszámítja a választ: $s = (k - c \cdot x) \bmod q$.
3. A t commitment és s response értékeket a `POST /api/auth/login/verify` végpontra küldi.
4. A szerver kiolvassa a kihívást (c) a `ChallengeStore`-ból és a verifier-t (y) az adatbázisból.
5. Ellenőrzi: $g^s \cdot y^c \equiv t \pmod{p}$.
6. Sikeres ellenőrzés esetén, amennyiben a felhasználónak nincs aktív TOTP beállítása, HS256-aláírt JWT tokenet állít ki 2 órás érvényességgel. Ha a TOTP aktív, egy ideiglenes `pending_token`-t ad vissza (lásd 7.2. alfejezet).

7.1.4. Hibakezelés és biztonság (US-15)

- **Fiókzárolás:** 5 sikertelen bejelentkezési kísérlet után a fiók ideiglenesen zárolódik (`locked_until` mező az adatbázisban). Sikeres bejelentkezés esetén a számláló nullázódik.
- **Egyszeri challenge:** Minden kiadási kihívás egyszeri felhasználású — a sikeres vagy sikertelen ellenőrzés után azonnal törlődik a `ChallengeStore`-ból.
- **Verifier validáció:** A regisztrációkor a szerver ellenőrzi, hogy a verifier érvényes csoportelem-e (nem nulla, hexadecimális formátum).

7.1.5. A Schnorr-ellenőrzés implementációja

Az alábbi kódrészlet a szerver-oldali bizonyítás-ellenőrzés Go implementációját mutatja be (`internal/crypto/schnorr.go`). A függvény a $g^s \cdot y^c \equiv t \pmod{p}$ egyenlőség teljesülését vizsgálja:

```

1 func (gp *GroupParams) VerifyProof(
2     verifier, commitment, challenge, response *big.Int,
3 ) bool {
4     if !gp.IsValidGroupElement(verifier) ||
5         !gp.IsValidGroupElement(commitment) {
6         return false
7     }
8     // g^s mod p
9     gs := new(big.Int).Exp(gp.G, response, gp.P)
10    // y^c mod p
11    vc := new(big.Int).Exp(verifier, challenge, gp.P)
12    // g^s * y^c mod p
13    lhs := new(big.Int).Mul(gs, vc)
14    lhs.Mod(lhs, gp.P)
15
16    return lhs.Cmp(commitment) == 0
17 }

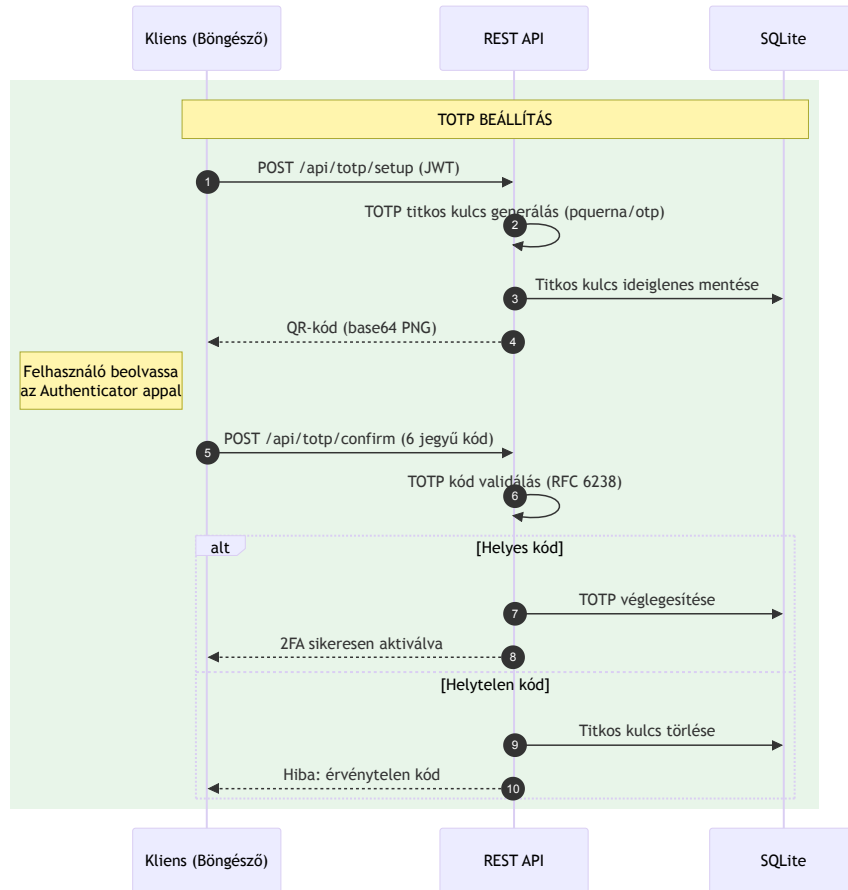
```

7.2. forráskód. A Schnorr-bizonyítás szerver-oldali ellenőrzése (`schnorr.go`)

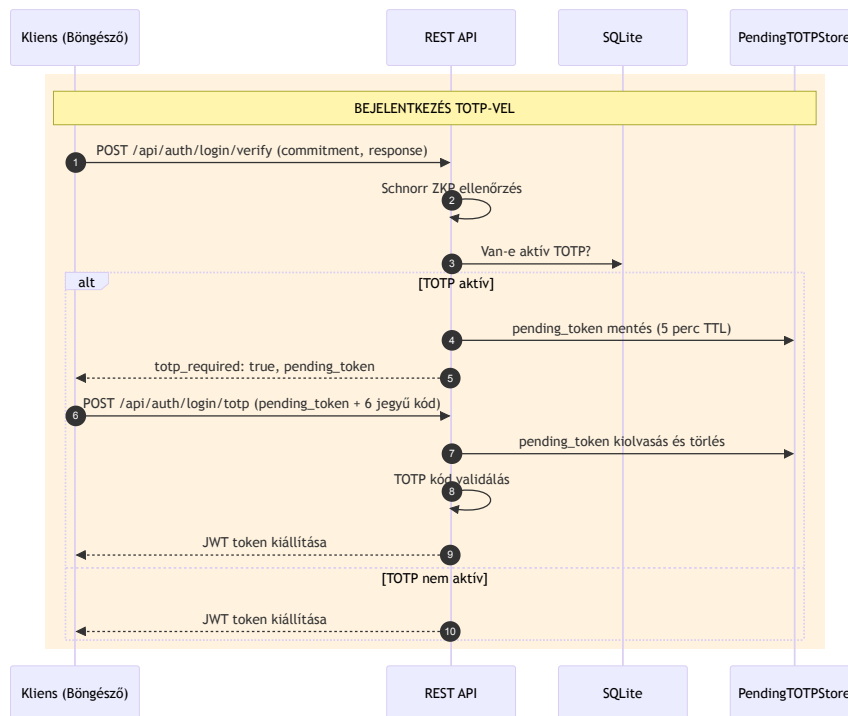
A függvény először ellenőrzi, hogy a verifier és a commitment érvényes csoportelemek-e, majd kiszámítja a $g^s \pmod{p}$ és $y^c \pmod{p}$ értékeket, végül összehasonlítja szorzatukat a kapott commitment-tel. A Go `math/big` csomag `Exp` metódusa hatékony moduláris hatványozást végez (square-and-multiply algoritmus).

7.2. TOTP kétfaktoros azonosítás

A ZKP-alapú hitelesítés önmagában is erős védelmet nyújt, de a rendszer egy második védelmi réteggel is kiegészíthető: TOTP (Time-Based One-Time Password) [3] alapú kétfaktoros azonosítással (2FA). A TOTP az HOTP (HMAC-Based One-Time Password) [35] algoritmuson alapul, időbélyeggel kiegészítve. A beállítási folyamatot a 7.1. ábra, a TOTP-vel történő bejelentkezést a 7.2. ábra szemlélteti.



7.1. ábra. A TOTP kétfaktoros azonosítás beállítási folyamata



7.2. ábra. Bejelentkezés TOTP kétfaktoros azonosítással

7.2.1. TOTP beállítás (US-04)

1. A felhasználó a chat oldalon megnyitja a beállítások modalt, és a `POST /api/totp/setup` végpontra küld kérést (JWT hitelesítéssel).
2. A Go szerver a `pquerna/otp` [34] könyvtár segítségével generál egy TOTP titkos kulcsot.
3. A titkos kulcsot QR-kód formájában (base64 PNG) prezentálja a felhasználónak, aki mobil autentikátor alkalmazással (pl. Google Authenticator, Authy) beolvassa.
4. A titkos kulcs ideiglenesen az adatbázisban tárolódik.
5. A felhasználónak megerősítésként be kell írnia az első 6 jegyű kódot a `POST /api/totp/confirm` végponton. Helytelen kód esetén a titkos kulcs törlődik, és a beállítást újra kell kezdeni.

7.2.2. TOTP ellenőrzés (US-05)

Bejelentkezéskor, sikeres ZKP ellenőrzést követően:

1. A szerver ellenőrzi, hogy a felhasználónak aktív TOTP beállítása van-e.
2. Amennyiben igen, a szerver *nem* állít ki JWT token, hanem egy ideiglenes `pending_token`-t generál (5 perces érvényességgel, egyszeri felhasználású), amelyet memóriabeli `PendingTOTPStore`-ban tárol.
3. A kliens megjelenít egy 6 jegyű, időalapú kódot kérő mezőt.
4. A kliens a kódot és a `pending_token`-t a `POST /api/auth/login/totp` végpontra küldi.
5. A szerver az RFC 6238 [3] szabvány szerint validálja a kódot (30 másodperces ablakkal).
6. A JWT token a szerver kizárólag sikeres TOTP ellenőrzés után állítja ki.

7.2.3. TOTP kikapcsolás (US-14)

A felhasználó a beállítások modálban deaktiválhatja a 2FA funkciót a `POST /api/totp/disable` végponton. Biztonsági okokból a művelethez meg kell adnia az aktuálisan érvényes 6 jegyű TOTP kódot. Sikeres validálás után a tárolt titkos kulcs törlődik az adatbázisból.

7.3. Chat funkció

A hitelesítés után a felhasználó a valós idejű csevegőfelületre kerül. A kommunikáció a WebSocket protokollra [30] épül, az implementáció a `gorilla/websocket` [29] könyvtárat alkalmazza a kapcsolatkezeléshez, és egy központi Hub architektúrális mintát követ az üzenettovábbításhoz.

7.3.1. Hub architektúra

A rendszer üzenettovábbítási magja a Hub struktúra (`internal/chat/hub.go`), amely az alábbi feladatokat látja el:

- Az aktív WebSocket kapcsolatok nyilvántartása (`sync.RWMutex`-védelemmel a párhuzamos hozzáférés miatt).
- Felhasználónév alapú üzenettovábbítás (routing) a megfelelő kliensre.
- Jelenlét-információ (presence) szórása: új felhasználó csatlakozásakor vagy kilépésekor minden aktív kliens értesítést kap.
- Offline üzenetek kézbesítése: a korábban nem kézbesített üzenetek automatikus elküldése a csatlakozás pillanatában.

A kliens–szerver közötti kommunikáció egységes JSON *envelope* formátumot használ:

```
1 {  
2     "type":      "message" | "presence" | "history" | "error",  
3     "from":      "felado felhasznalonev",  
4     "to":        "cimzett felhasznalonev",  
5     "content":   "uzenet szovege",  
6     "users":     [{"username": "...", "online": true/false}],  
7     "messages":  [...],  
8     "error":     "hibauzenet",  
9     "time":      "2025-01-15T10:30:00Z"  
10 }
```

7.3. forráskód. A WebSocket üzenet envelope formátuma (JSON)

Az `omitempty` JSON tag-ek biztosítják, hogy az üres mezők ne kerüljenek bele az átvitt adatsomagba (sávszélesség-optimalizáció).

7.3.2. WebSocket kapcsolat és hitelesítés (US-16)

A WebSocket handshake a `GET /ws?token=<JWT>` végponton keresztül történik. A Handler struktúra (`internal/chat/handler.go`) a következő lépéseket hajtja végre:

1. A JWT token kiolvasása a query paraméterből.
2. A token aláírásának (HMAC-SHA256) és érvényességi idejének ellenőrzése.
3. A `sub` (subject) claim-ből a felhasználónév kinyerése.
4. A HTTP kapcsolat WebSocket protokollra váltása (Upgrade).
5. A felhasználó regisztrálása a Hub-ban. Ha a felhasználónak korábban volt aktív kapcsolata, a Hub a régit lezárja.
6. Jelenlét-broadcast küldése minden aktív kliensnek.
7. A kézbesítetlen (offline) üzenetek automatikus elküldése.

A kapcsolat hitelesítése query paraméteren keresztül történik (a WebSocket protokoll nem támogatja a custom fejléceket a böngészős JavaScript API-ban), de a token ugyanolyan HMAC-SHA256 validáción esik át, mint a REST API végpontoknál.

7.3.3. Felhasználók listázása (US-11)

A `GET /api/users` REST végpont (JWT-hitelesítéssel védett) visszaadja az összes regisztrált felhasználó nevét. A WebSocket csatornán emellett a **presence** típusú envelope valós időben jelzi az online/offline állapotváltozásokat.

7.3.4. Üzenet küldése és fogadása (US-06, US-07)

Az üzenetküldés folyamata:

1. A kliens (`chat.js` modul) egy **message** típusú envelope-ot küld a WebSocket csatornán, amely tartalmazza a címzett felhasználónevét (**to**) és az üzenet szövegét (**content**).
2. A Hub a `HandleMessage` metódusban validálja az üzenetet (nem üres tartalom, létező címzett).
3. A `MessageRepository` (`internal/repository/message.go`) az üzenetet elmenti az SQLite `messages` táblájába.
4. Amennyiben a címzett rendelkezik aktív WebSocket kapcsolattal, a Hub azonnal továbbítja az üzenetet a dedikált `send` csatornáján (Go channel).
5. Ha a címzett offline, az üzenetet a szerver `delivered = 0` állapottal menti, és a címzett következő csatlakozásakor automatikusan kézbesíti.

Minden kliens számára egy dedikált goroutine (`writePump`) felel az üzenetek pufferekt kiírásáért, amely elkerüli a blokkolódást a Hub fő logikájában.

Az alábbi kódrészlet a Hub üzenettovábbítási logikájának központi részét mutatja be (`internal/chat/hub.go`), amely az online címzettnek történő azonnali kézbesítést valósítja meg:

```

1 // Deliver to recipient if online.
2 h.mu.RLock()
3 recipient, online := h.clients[env.To]
4 h.mu.RUnlock()
5
6 if online {
7     select {
8     case recipient.send <- data:
9         _ = h.messageRepo.MarkDelivered([]int64{msgID})
10    default:
11    }
12 }

```

7.4. forráskód. Az online címzettnek történő üzenettovábbítás (`hub.go`)

A `sync.RWMutex` olvasási zárral biztosítja a szálbiztos hozzáférést a kliensek map-jéhez. A Go `select` utasítás nem blokkoló küldést valósít meg: ha a kliens `send` csatornája megtelt, az üzenet nem vész el (az adatbázisban `delivered = 0` marad), elkerülve a Hub fő goroutine-jának blokkolódását.

7.3.5. Korábbi üzenetek (US-12)

A `GET /api/messages/{peer}` REST végpont lapozható (limit/offset) üzenet-előzményeket szolgáltat két felhasználó között. A `repository.MessageRepository.GetHistory` metódus SQL JOIN-okkal kéri le a feladó és a címzett felhasználónevét, így a kliens közvetlenül megjeleníthető adatokat kap.

7.3.6. Offline üzenetek (US-17)

A `Hub.deliverUnread` metódus a felhasználó WebSocket csatlakozásakor automatikusan:

1. Lekéri az összes kézbesítetlen üzenetet a `MessageRepository.GetUndelivered` metódussal.

2. Egyetlen `history` típusú `envelope`-ban elküldi azokat a kliensnek.
3. A `MessageRepository.MarkDelivered` metódussal kézbesítettnek jelöli az üzenet-azonosítókat.

7.3.7. Kliens-oldali chat modul

A `chat.js` JavaScript modul (IIFE mintával) a következő funkciókat látja el:

- Automatikus újracsatlakozás: ha a `WebSocket` kapcsolat megszakad, a modul exponenciális visszalépéssel (`backoff`) újrapróbálkozik.
- Felhasználólista és jelenlétjelző: a `presence` `envelope`-ok alapján valós időben frissíti az oldalsávban megjelenő felhasználólistát (zöld/szürke jelző).
- Üzenet-buborékok: a küldött és fogadott üzenetek megkülönböztetett vizuális megjelenítéssel (eltérő szín, igazítás) jelennek meg.
- XSS-védelem: a `escapeHtml` segédfüggvény megelőzi a cross-site scripting támadásokat az üzenetek megjelenítésekor.

7.4. Jelszóvisszaállítás és session kezelés

Egy ZKP-alapú rendszerben a jelszóvisszaállítás sajátos kihívás, hiszen a szerver nem rendelkezik a jelszóval. A megoldás egy `recovery` kulcsra épülő visszaállítási mechanizmus és a hozzá kapcsolódó `session`-életciklus kezelés.

7.4.1. Recovery key generálás (US-13)

A regisztrációs folyamat során a kliens automatikusan generál egy 256 bites (32 byte-os) `recovery` kulcsot, amelynek a szerepe megegyezik egy tartalék jelszóéval:

1. A kliens véletlenszerűen generálja a `recovery` kulcsot (`crypto.getRandomValues`).
2. A normál jelszóval azonos `PBKDF2-SHA256` kulcsszármaztatással kiszámítja a `recovery` titkos kulcsot: $x_r = \text{KDF}(\text{recovery_key}, \text{salt})$.
3. Kiszámítja a `recovery` verifier-t: $y_r = g^{x_r} \bmod p$.

4. A y_r értéket elküldi a szervernek a regisztrációs kérés részeként.
5. A szerver a `recovery_verifier` oszlopban tárolja az adatbázisban.
6. A recovery kulcs hexadecimális formátumban **egyszer** jelenik meg a felhasználó számára, másolható a vágólapra. A szerver soha nem látja a nyers kulcsot.

7.4.2. Jelszóvisszaállítás (US-08, US-09)

A jelszóvisszaállítás a normál bejelentkezéssel azonos Schnorr ZKP protokollt alkalmazza, de a recovery verifier (y_r) ellen:

1. A felhasználó a bejelentkezési oldalon az „Elfelejtett jelszó” linkre kattint, és megadja a felhasználónevét és a recovery kulcsot.
2. A kliens a `POST /api/auth/recovery/init` végponton kér salt-ot és challenge-et. A szerver a `recovery:` prefixszel tárolja a challenge-et a `ChallengeStore`-ban.
3. A kliens a tárolt salt-tal kiszámítja $x_r = \text{KDF}(\text{recovery_key}, \text{salt})$, majd a szokásos Schnorr-bizonyítást végzi el: $t = g^k \bmod p$, $s = (k - c \cdot x_r) \bmod q$.
4. A kliens generálja az új jelszóhoz tartozó credential-öket (új salt, új verifier) és egy új recovery kulcsot (új recovery verifier).
5. A `POST /api/auth/recovery/reset` végpontra elküldi a Schnorr-bizonyítást és az új credential-öket.
6. A szerver ellenőrzi: $g^s \cdot y_r^c \equiv t \pmod{p}$. Sikeres ellenőrzés esetén felülírja az adatbázisban a salt-ot, verifier-t és a recovery verifier-t.
7. A régi jelszó és a régi recovery kulcs érvénytelenné válik. Az új recovery kulcs megjelenik a felhasználó számára.

A recovery folyamat során is érvényesül a fiókszárolás: 5 sikertelen recovery kísérlet után a fiók 15 percre zárolódik.

7.4.3. Kijelentkezés (US-10)

A kijelentkezés a kliens-oldali JWT token törlését és a WebSocket kapcsolat bontását jelenti. A `chat.js` modul `disconnect()` metódusa bontja a WebSocket kapcsolatot, leállítja az automatikus újracsatlakozást és a token-frissítési időzítőt, majd a `localStorage`-ból törli a JWT token és a felhasználónevet.

7.4.4. Token lejárat és session kezelés (US-18)

A session-életciklus a következő mechanizmusokra épül:

- **Rövidített token-érvényesség:** A JWT tokenek 2 órás érvényességi idővel rendelkeznek (a korábbi 24 óra helyett), csökkentve a kompromittált token kihasználhatósági ablakát.
- **Automatikus token-frissítés:** A `POST /api/auth/refresh` végpont (JWT-hitelesítéssel védett) új token állít ki a jelenlegi érvényes token alapján. A kliens 5 perccel a lejárat előtt automatikusan frissíti a token. A handler az `AuthMiddleware` által a kontextusba írt felhasználónevet a `middleware.UserContextKey` típusspecifikus kulccsal olvassa ki, mert a Go kontextus típus- és értékfüggő összehasonlítása miatt `plain string` kulcs nem adna vissza egyező értéket.
- **WebSocket heartbeat:** A szerver 30 másodpercenként WebSocket ping üzenetet küld minden aktív kliensnek. A kliens pong válaszát a szerver 60 másodperces időablakkal várja — ennek lejártakor a kapcsolat megszakad.
- **Automatikus lecsatlakoztatás:** A szerver a `writePump` goroutine-ban nyomon követi a kliens JWT token lejáratát. Ha a token lejár, a szerver `ClosePolicyViolation` (1008-as kód) WebSocket close frame-mel bontja a kapcsolatot. A kliens megpróbálja frissíteni a token és újracsatlakozni; ha ez sikertelen, átirányítja a felhasználót a bejelentkezési oldalra.

7.5. Alternatív telepítés: Docker konténer

A 6. fejezetben bemutatott natív telepítési mód (`go run / make run`) mellett a rendszer Docker konténerben is futtatható, így a gazdarendszerre nem kell Go

fordítót vagy más fejlesztői eszközt telepíteni: elég egyetlen Docker runtime (Docker Engine vagy Podman), amely izolált és reprodukálható környezetet ad.

7.5.1. Image felépítése

A `zkip-chat/Dockerfile` kétlépcsős (multi-stage) felépítést alkalmaz, amely jelentősen csökkenti a végleges image méretét és az esetleges támadási felületet:

1. **Builder réteg** (`golang:1.24-alpine`): a Go forráskód lefordítása statikusan linkelt binárisra (`CGO_ENABLED=0, -trimpath, -ldflags="-s -w"`). A statikus linkelés azért valósítható meg, mert a `modernc.org/sqlite` [33] tiszta Go-ban implementált SQLite motor, így nincs szükség külső C könyvtárakra vagy rendszerszintű `libsqlite3` csomagra.
2. **Futtató réteg** (`alpine:3.20`): csak a lefordított binárist és a `web/` statikus erőforrásokat (HTML sablonok, CSS, JavaScript) tartalmazza. A folyamat nem `root`, hanem egy dedikált, nem rendszergazdai jogú (`app`, UID 10001) felhasználóval fut, a legkisebb jogosultság (least privilege) elve szerint.

A Dockerfile váza a 7.5. listában látható.

```
1 # --- Build stage ---
2 FROM golang:1.24-alpine AS builder
3 WORKDIR /src
4 COPY go.mod go.sum ./
5 RUN go mod download
6 COPY cmd ./cmd
7 COPY internal ./internal
8 RUN CGO_ENABLED=0 GOOS=linux go build \
9     -trimpath -ldflags="-s -w" -o /out/server ./cmd/server
10
11 # --- Runtime stage ---
12 FROM alpine:3.20
13 RUN adduser -D -u 10001 app
14 WORKDIR /app
15 COPY --from=builder /out/server /app/server
16 COPY web ./web
17 RUN mkdir -p /data && chown -R app:app /data /app
18 USER app
19 ENV PORT=8080 \
20     DB_PATH=/data/zkp-chat.db \
21     PARAMS_PATH=/data/params.json
22 EXPOSE 8080
23 VOLUME ["/data"]
24 ENTRYPOINT ["/app/server"]
```

7.5. forráskód. A kétlépcsős Dockerfile váza

7.5.2. Perzisztencia és konfiguráció

A konténer saját fájlrendszere ephemer: minden `docker rm` vagy image-frissítés után elvész, ezért a szerver perzisztens állapotát külső volume-ra kell helyezni. A rendszer perzisztens állapota két fájlra korlátozódik:

- `zkp-chat.db` — az SQLite adatbázis (felhasználók, recovery verifier-ek, TOTP titkok, üzenetek).

- `params.json` — a 2048 bites Schnorr csoport generált paraméterei (p , q , g). Első indításkor percekig tartó művelet, ezért indokolt egyszer generálni és megőrizni, így minden felhasználó azonos paraméterek ellen bizonyít.

Mindkét fájl a konténeren belüli `/data` könyvtárba kerül, amely Docker *named volume*-ként (`zkp-chat-data`) van bekötve. Ezt a `VOLUME ["/data"]` direktíva deklarálja, és a `DB_PATH`, illetve `PARAMS_PATH` környezeti változók irányítják a `config.Load` függvényt (`internal/config/config.go`) a helyes elérési utakra. A `JWT_SECRET` ugyancsak környezeti változóból olvasódik, így éles környezetben futtatáskor kötelező titkos értékre cserélni.

7.5.3. Orkesztráció Docker Compose-szal

A futtatási topológiát a `docker-compose.yml` írja le. A fájl a kézi `docker run` parancsnál verziózható és reprodukálható indítási módot ad, és az egyetlen szolgáltatás, a `zkp-chat` köré szerveződik:

```
1 services:
2   zkp-chat:
3     build: .
4     image: zkp-chat:latest
5     container_name: zkp-chat
6     ports:
7       - "8080:8080"
8     environment:
9       PORT: "8080"
10      DB_PATH: /data/zkp-chat.db
11      PARAMS_PATH: /data/params.json
12      JWT_SECRET: ${JWT_SECRET:-dev-secret-change-in-
13                  production}
14      volumes:
15        - zkp-chat-data:/data
16      restart: unless-stopped
17 volumes:
18   zkp-chat-data:
```

7.6. forráskód. `docker-compose.yml` — a teljes futtatási topológia

A `restart: unless-stopped` szabály biztosítja, hogy a konténer a gazdagép újraindulása után vagy váratlan hiba esetén automatikusan újrainduljon, kivéve ha a rendszergazda kifejezetten leállította. A `JWT_SECRET` a shell-környezetből veszi át az értéket (`$JWT_SECRET:-...` szintaxis), így a titkot nem kell a verziókövetett fájlba beírni.

7.5.4. Futtatási workflow

A Docker-alapú telepítés kényelmes használatához a `Makefile` dedikált célokat biztosít, amelyek egyesítik az image-készítés és az indítás lépéseit:

- `make docker-build` — a production image (`zkp-chat:latest`) elkészítése a `Dockerfile` alapján.
- `make docker-run` — az image egyszeri futtatása (`docker run`) port-továbbítással és a `zkp-chat-data` volume csatolásával.
- `make docker-up` — a teljes compose-stack háttérben történő elindítása (`docker compose up -d`), szükség esetén újraépítéssel.
- `make docker-down` — a compose-stack leállítása és eltávolítása. Az adatkötet megmarad, így a `docker-up` újbóli hívásakor a teljes állapot (felhasználók, üzenetek, Schnorr paraméterek) elérhető.

A Docker-alapú futtatás funkcionálisan ugyanazt az eredményt adja, mint a 6. fejezet natív `make run` indítása, azzal a különbséggel, hogy a gazdagépen egyetlen függőség marad, a Docker runtime. Ez szervertelepítéskor és a tesztelési (8.) fejezet E2E futtatási scénárióinak egységes környezetben történő reprodukálásakor egyaránt előnyös.

8. fejezet

Tesztelés

A kriptográfiai rendszerek tesztelése különös figyelmet igényel: egy elírás a moduláris aritmetikában vagy egy rosszul kezelt szélsőérték a protokoll egész biztonságát alááshatja. A tesztkészletet ennek tudatában három szintű (a szoftvertesztelési szakirodalomban *tesztpiramisnak* [36, 37] nevezett) rétegzés mentén építettem fel. A rétegek elnevezését és megközelítését a hivatalos Android tesztelési dokumentáció [38] inspirálta: ott is három réteg (*local unit tests*, *instrumented tests*, *UI tests*) különül el hasonló arányokkal és hasonló felelősségekkel.

8.1. A tesztpiramis alkalmazása

A piramis logikáját az alábbi alapelv vezérli: minél gyorsabb és izoláltabb egy teszt, annál többet kell belőle írni; minél lassabb és több komponenst vonultat fel, annál kevesebbet. Ennek megfelelően a tesztkészlet három rétegre oszlik, amelyek futási sebességben és függőségeik körében nagyságrendileg különböznek egymástól.

Réteg	Android megfelelő [38]	Függőségek	Tipikus idő
Egységtesztek (unit)	Local unit tests (JVM)	nincs I/O, mock repo	< 1 ms/eset
Integrációs (instrumented)	Instrumented tests	in-memory SQLite, <code>httptest</code>	< 500 ms/eset
End-to-end (E2E)	UI / Espresso	teljes szerver, WebSocket	1–30 s/eset

8.1. táblázat. A három tesztréteg és az Android világban használt megfelelőik

A rétegek fizikailag Go *build tag*-ekkel vannak elkülönítve: minden tesztfájl vagy `//go:build unit`, `//go:build integration`, vagy `//go:build e2e` direktívát visel (kivéve a tiszta, I/O nélküli egységteszteket, amelyek mindhárom rétegben futhatnak). A Makefile három külön célt biztosít a rétegek futtatására: `make`

`test-unit` (gyors visszajelzés fejlesztés közben), `make test-integration` (középréteg, valódi SQLite), és `make test-e2e` (teljes rendszer). A `make test` mindhármat szekvenciálisan végrehajtja a CI pipeline-hoz.

Egy külön `make test-race` cél a Go versenyállapot-detektorát (`-race`) futtatja a konkurens komponensek (WebSocket Hub, repository réteg) tesztjein, ami elengedhetetlen egy többszálú kiszolgáló megbízhatóságához.

A tesztek kizárólag a Go nyelv beépített eszközeire támaszkodnak: a `testing` standard könyvtári csomag biztosítja a tesztvázatot, a `net/http/httpptest` csomag a HTTP handler tesztek szimulált kérés-válasz környezetét, a `gorilla/websocket` könyvtár pedig a WebSocket kapcsolati tesztek kliens-oldalát. Külső tesztelési keretrendszer (pl. `testify`) nem szükséges. A tesztkörnyezetben kisméretű, 64 bites Schnorr-csoportparamétereket alkalmazok a gyors végrehajtás érdekében.

Réteg	Tesztek száma	Célarány (Google/Fowler [37])
Egységtesztek	54	~70 %
Integrációs tesztek	112	~20 %
End-to-end tesztek	20	~10 %
Összesen	186	–

8.2. táblázat. A tesztkészlet rétegenkénti megoszlása

A projektben az integrációs réteg a céltól némileg nagyobb, mert a kriptográfiai és autentikációs logika kevés izolált egységre bontható, és minden lényeges ellenőrzés valódi adatbázison keresztül történik. Ez a hangsúlyeltolás szándékos: inkább kevesebb, de érdemi teszt fut le, mint sok olyan, amely mockok mögött nem fog valós hibát.

8.2. 1. réteg: Egységtesztek

Az első réteg tesztjei semmilyen I/O-t nem végeznek: sem fájlrendszert, sem hálózatot, sem adatbázist nem érintenek. A külső függőségeket interfészek és kézzel írt *teszt dublőrök* (test doubles) helyettesítik. Ennek köszönhetően egy-egy eset mikroszekundumok alatt lefut, és a teljes egységteszt-készlet kevesebb mint másodperc alatt végez.

8.2.1. Mock repository és service-szintű hibaág-tesztek

A `repository` csomagban definiáltam a `UserRepo` és `MessageRepo` Go interfészeket, amelyek a konkrét (SQLite-ot használó) implementációk publikus API-ját deklarálják. A `service` réteg konstruktorai ezentúl az interfészeket fogadják, ami lehetővé teszi, hogy a teszteknel egy kézzel írt mock-ot adjunk át, és minden repository metódus viselkedése külön beállítható legyen. Erre azért van szükség, mert a korábbi, valódi adatbázisra épülő tesztek nem tudták kiváltani az `error` ágakat (pl. „mi történik, ha a lemez megtelt a `CreateUser` közben?”); ezeket az ágakat most a mock tetszőleges hibával tudja aktiválni.

Teszt eset	Leírás	Eredmény
TestRegister_CreateFails_ReturnsError	Mock <code>Create</code> hibát ad → hiba tovább	Elutasítva
TestRegister_ExistsCheckFails_ReturnsError	<code>Exists</code> lekérdezés hibája → propagál	Elutasítva
TestLoginInit_GetByUsernameFails_ReturnsUserNotFound	DB hiba → <code>ErrUserNotFound</code> (anti-enum.)	Elutasítva
TestLoginInit_LockedAccount_ReturnsAccountLocked	Zárolt fiók → <code>ErrAccountLocked</code>	Zárolva
TestLoginVerify_GetByUsernameFails_ReturnsInvalidProof	DB hiba → <code>ErrInvalidProof</code> (ne szivároogjon)	Elutasítva
TestRecoveryInit_NoRecoveryVerifier_ReturnsError	Recovery kulcs nélkül → hiba	Elutasítva
TestRecoveryReset_GetByUsernameFails_ReturnsProofFailed	DB hiba → <code>ErrRecoveryProofFailed</code>	Elutasítva

8.3. táblázat. AuthService mock alapú egységtesztek (`auth_unit_test.go` — 7 teszt)

Teszteteset	Leírás	Eredmény
TestSetupTOTP_ GetByUsernameFails_ ReturnsUserNotFound	DB hiba a setup lekérdezés közben	Elutasítva
TestSetupTOTP_ AlreadyEnabled_ReturnsError	Aktív TOTP újrakonfigurálás	Elutasítva
TestSetupTOTP_ SetTOTPSecretFails_ ReturnsError	Perzisztálási hiba terjedjen	Elutasítva
TestConfirmTOTP_ InvalidCode_ClearsSecret	Rossz kód → secret törlés (mock ellenőriz)	Elutasítva
TestVerifyLoginTOTP_ PendingTokenMissing_ ReturnsExpired	Ismeretlen/lejárt pending token	Elutasítva
TestVerifyLoginTOTP_ InvalidCode_ LocksAfterFiveFailures	5. sikertelen TOTP → fiókszár (mock ellenőriz)	Zárolva
TestDisableTOTP_ NotEnabled_ReturnsError	Kikapcsolás TOTP nélkül	Elutasítva
TestHasTOTP_ Enabled/Disabled	TOTP státusz ellenőrzése mock-kal	Sikeres

8.4. táblázat. TOTPService mock alapú egységtesztek (`totp_unit_test.go` — részlet a 14-ből)

8.2.2. Crypto egységtesztek

A Schnorr ZKP matematikai korrektségét vizsgáló tesztek nem igényelnek semmilyen függőséget, csak tiszta `math/big` aritmetikát. Ezeket a tesztek minden rétegben megjelenítik (nincs build tag), de természetes helyük az egység réteg.

Teszt eset	Leírás	Eredmény
TestGenerateGroupParams	p, q prímek; $q \mid p - 1$; $g^q \equiv 1$	Sikeres
TestVerifyProof_Valid	Helyes titkos kulccsal generált proof	Sikeres
TestVerifyProof_Invalid	Rossz titkos kulccsal generált proof	Sikeres
TestIsValidGroupElement	Csoporttagság-ellenőrzés	Sikeres
TestFullProtocolFlow	Teljes regisztráció $\rightarrow$ login szimuláció	Sikeres
TestMultipleProtocolRuns	10 $\times$ egymás utáni login	Sikeres
TestVerifyProof_SecretKeyIsOne	Speciális eset: $x = 1$	Sikeres
TestVerifyProof_SecretKeyIsQMinusOne	Speciális eset: $x = q - 1$	Sikeres
TestVerifyProof_ZeroResponse	Nulla response elutasítása	Sikeres
TestVerifyProof_ReplayedProof	Replay-támadás detektálása	Sikeres

8.5. táblázat. A Schnorr crypto tesztek (részlet a 18-ból)

8.2.3. Middleware és envelope tesztek

A JWT autentikációs middleware külön tesztelési figyelmet igényel, hiszen ez a réteg védi az összes hitelesítést igénylő végpontot. A tesztek egy *sentinel* handler segítségével ellenőrzik, hogy a middleware megfelelő esetekben továbbhív-e a következő kezelőre, illetve blokkolja-e az érvénytelen kéréseket. A Hub réteg envelope JSON-serializációs tesztjei szintén ebbe a rétegbe tartoznak: tisztán memóriában zajló `encoding/json` műveletek.

Teszt eset	Leírás	Eredmény
TestAuthenticate_ValidToken_Passes	Érvényes JWT → átengedi	200
TestAuthenticate_ValidToken_InjectsUsernameIntoContext	Felhasználónév kontextusba írva	200
TestAuthenticate_MissingAuthHeader_Returns401	Hiányzó Authorization fejléc	401
TestAuthenticate_NotBearerFormat_Returns401	Nem Bearer séma	401
TestAuthenticate_MalformedToken_Returns401	Hibás token formátum	401
TestAuthenticate_ExpiredToken_Returns401	Lejárt token	401
TestAuthenticate_WrongSigningSecret_Returns401	Helytelen aláírási kulcs	401
TestAuthenticate_Failure_DoesNotCallNextHandler	Sikertelen auth nem hív tovább	ellenőrzött

8.6. táblázat. Az autentikációs middleware tesztek

8.3. 2. réteg: Integrációs tesztek

A második réteg tesztjei valódi komponenseket használnak (in-memory SQLite adatbázist a `:memory:` DSN-nel, valódi HTTP kérést `httptest.NewServer`-en keresztül és valódi WebSocket kapcsolatot helyi TCP-loopbacken), de nem érintenek külső szolgáltatást vagy perzisztens lemezt. Ez megfelel az Android *instrumented test* kategóriájának [38]: a tesztelendő rendszer úgy fut, ahogy produkcióban, csak a „külvilág” (a lemez és a hálózat) cserélődik le ekvivalens, de gyorsabb változatra.

8.3.1. ZKP autentikációs folyamat

Az auth service integrációs tesztjei a teljes regisztráció–login–recovery folyamatot valódi SQLite-on végigfuttatják. A fiókzárolás és a challenge egyszeri felhasználhatósága itt nem izolált mock-viselkedést, hanem a valódi DB állapotátmeneteket ellenőrzi.

Teszt eset	Leírás	Eredmény
TestRegister__Success	Sikeres regisztráció érvényes verifier-rel	Sikeres
TestRegister__DuplicateUsername	Duplikált felhasználónév	Elutasítva
TestRegister__InvalidVerifier	Érvénytelen verifier formátum	Elutasítva
TestRegister__VerifierIsZero	Nulla értékű verifier	Elutasítva
TestLoginFlow__Success	Teljes ZKP login flow	JWT kiállítva
TestLoginFlow__InvalidProof	Hibás proof	Elutasítva
TestLoginVerify__NoChallenge	Challenge nélküli verify	Elutasítva
TestLoginVerify__ChallengeIsOneTimeUse	Challenge replay-védelem	Elutasítva
TestAccountLocking__After5FailedAttempts	5 sikertelen → fiókszárolás	Zárolva
TestSuccessfulLogin-ResetsFailedAttempts	Számláló nullázás	Sikeres

8.7. táblázat. Az auth service integrációs tesztek (részlet a 15-ből)

Teszt eset	Leírás	Eredmény
TestSetupTOTP_Success	TOTP secret + QR-kód generálás	Sikeres
TestSetupTOTP_AlreadyEnabled	Dupla setup → hiba	Elutasítva
TestConfirmTOTP_ValidCode	Megerősítés helyes kóddal	Sikeres
TestConfirmTOTP_InvalidCode_ClearsSecret	Rossz kód → secret törlés	Elutasítva
TestLogin_WithTOTP_ReturnsErrTOTPRequired	ZKP OK + TOTP aktív → pending token	Sikeres
TestVerifyLoginTOTP_Success	Érvényes TOTP kód → JWT	Sikeres
TestVerifyLoginTOTP_InvalidCode	Rossz kód → elutasítás	Elutasítva
TestVerifyLoginTOTP_ExpiredPendingToken	Lejárt pending token	Elutasítva
TestVerifyLoginTOTP_Pending-TokenIsOneTimeUse	Token replay-védelem	Elutasítva
TestDisableTOTP_Success	Kikapcsolás helyes kóddal	Sikeres
TestDisableTOTP_InvalidCode	Kikapcsolás rossz kóddal	Elutasítva
TestFullFlow_Register-EnableTOTPLoginDisable	Teljes E2E: regisztráció → 2FA → login → disable	Sikeres

8.8. táblázat. A TOTP service integrációs tesztek (részlet a 16-ból)

Teszt eset	Leírás	Eredmény
TestRecoveryInit_Success	Recovery init: salt + challenge visszaadása	Sikeres
TestRecoveryInit_NonexistentUser	Nem létező felhasználó → hiba	Elutasítva
TestRecoveryReset_Success	Helyes recovery key → új jelszó beállítása	Sikeres
TestRecoveryReset_InvalidProof	Hibás recovery key → elutasítás	Elutasítva
TestRecoveryReset_NoChallenge	Challenge nélküli reset → elutasítás	Elutasítva
TestRecoveryReset_ChallengeIsOneTimeUse	Challenge replay-védelem	Elutasítva
TestRecoveryReset_OldPasswordFails	Régi jelszó érvénytelen reset után	Sikeres
TestRecoveryReset_NewRecoveryKeyWorks	Új recovery key működik	Sikeres
TestRecoveryReset_OldRecoveryKeyFails	Régi recovery key érvénytelen	Elutasítva
TestRefreshToken_Success	Token frissítés → új JWT	Sikeres
TestRecoveryReset_AccountLocksAfter5Failures	5 sikertelen recovery → fiókszárolás	Zárolva

8.9. táblázat. A recovery service integrációs tesztek

8.3.2. Repository és HTTP handler tesztek

A `message_test.go` teszteli a valódi SQL JOIN alapú üzenetelőzmény-lekérdezést, a kézbesítetlen üzenetek szűrését és az ábécérendes felhasználólistát. A handler rétegben a tesztek `httptest.NewRecorder`-en keresztül szimulált HTTP kéréseket küldenek, és a visszakapott `http.ResponseRecorder` státusz kódját és JSON payloadját ellenőrzik. A tesztkörnyezet in-memory SQLite adatbázist és kisméretű Schnorr-paramétereket alkalmaz a gyors végrehajtás érdekében.

A handler tesztek megírása során két implementációs hibát sikerült azonosítani és javítani:

- **RefreshToken kontextuskulcs-hiba:** A `POST /api/auth/refresh` végpont kezelője a felhasználónevet plain `string` típusú kulccsal olvasta ki a Go kontextusból (`"username"`), miközben az `AuthMiddleware` egy típusspecifikus `contextKey("username")` kulccsal tárolta azt. A Go kontextus kulcsösszehasonlítása típus- és értékfüggő, ezért a lookup mindig sikertelen volt,

és minden tokenfrissítési kérés 401 `Unauthorized` választ kapott. A javítás: a handler a `middleware.UserContextKey` konstanst alkalmazza következetesen.

- **ListUsers null válasz:** A `GET /api/users` végpont üres felhasználói adatbázis esetén `{"users": null}` JSON választ adott vissza `{"users": []}` helyett. A Go `nil` szelet JSON-kódolása `null`-t eredményez, ami JavaScript kliensoldali hibát okozhat (`TypeError: Cannot read properties of null`). A javítás a `GetMessages` kezelőnél már alkalmazott `nil`-ellenőrzési mintát terjeszti ki a `ListUsers` kezelőre is.

Teszt eset	Leírás	Eredmény
TestMessageRepository_SaveAndGetHistory	Üzenet mentés + lekérdezés (JOIN)	Sikeres
TestMessageRepository_GetHistoryBidirectional	Kétirányú üzenet előzmények	Sikeres
TestMessageRepository_GetHistoryPagination	Limit + offset lapozás	Sikeres
TestMessageRepository_GetHistoryEmpty	Üres előzmény nem hibázik	Sikeres
TestMessageRepository_GetUndelivered	Kézbesítetlen üzenetek szűrése	Sikeres
TestMessageRepository_MarkDelivered	Kézbesítés jelölés + újralekérdezés	Sikeres
TestMessageRepository_MarkDeliveredEmpty	Üres/ <code>nil</code> slice nem hibázik	Sikeres
TestMessageRepository_MultipleUndelivered	Részleges kézbesítés	Sikeres
TestUserRepository_ListAll	Összes felhasználó listázása	Sikeres
TestUserRepository_GetIDByUsername	ID lekérdezés név alapján	Sikeres
TestUserRepository_GetIDByUsernameNotFound	Nem létező felhasználó → hiba	Sikeres

8.10. táblázat. Az üzenet repository tesztek

Teszt eset	Leírás	Eredmény
TestGetParams_ ReturnsGroupParams	p, q, g paraméterek visszaadása	200
TestRegister_ Success	Érvényes regisztrációs kérés	201
TestRegister_ MissingFields	Hiányos mezők	400
TestRegister_ DuplicateUsername	Foglalt felhasználónév	409
TestRegister_ InvalidVerifier	Érvénytelen hex verifier	400
TestLoginInit_ Success	Salt és challenge visszaadása	200
TestLoginInit_ MissingUsername	Hiányzó felhasználónév mező	400
TestLoginInit_ UnknownUser	Ismeretlen felhasználó (anti-enum.)	401
TestLoginVerify_ Success	Teljes ZKP bizonyítás $\rightarrow$ JWT	200
TestLoginVerify_ NoActiveSession	Challenge nélküli verify kísérlet	400
TestLoginVerify_ InvalidProof	Hibás Schnorr-bizonyítás	401
TestLoginVerify_ WithTOTP	ZKP OK + TOTP aktív $\rightarrow$ pending token	200
TestRecoveryInit_ Success	Recovery salt és challenge visszaadása	200
TestRecoveryReset_ MissingFields	Hiányos recovery reset kérés	400
TestRecoveryReset_ NoActiveSession	Challenge nélküli reset kísérlet	400
TestRefreshToken_ Success	Token frissítés (javított hiba)	200
TestRefreshToken_ MissingContext	Hitelesítés nélküli refresh	401

8.11. táblázat. Az auth handler tesztek (részlet a 26-ból)

Teszt eset	Leírás	Eredmény
TestListUsers_EmptyDB	Üres adatbázis → [] (nem null)	200
TestListUsers_WithRegisteredUsers	Regisztrált felhasználók listája	200
TestListUsers_OrderedAlphabetically	Ábécérendezett eredmény	200
TestGetMessages_EmptyHistory	Üres előzmény	200
TestGetMessages_PeerNotFound	Ismeretlen peer	404
TestGetMessages_WithMessages	Üzenetek lekérése	200
TestGetMessages_LimitParam	?limit=2 hatása az eredményre	200
TestGetMessages_OffsetParam	?offset=2 lapozás	200
TestGetMessages_BothDirections	Kétirányú üzenet-előzmény	200
TestGetMessages_InvalidLimit	Érvénytelen limit → alapértelmezett	200

8.12. táblázat. A chat handler tesztek

Teszt eset	Leírás	Eredmény
TestTOTPStatus_ NotEnabled	Új felhasználó: TOTP inaktív	200
TestTOTPStatus_ Enabled	Beállítás után: TOTP aktív	200
TestTOTPSetup_ Success	TOTP secret és QR-kód generálása	200
TestTOTPSetup_ AlreadyEnabled	Dupla beállítás kísérlete	409
TestTOTPConfirm_ Success	Megerősítés érvényes kóddal	200
TestTOTPConfirm_ MissingCode	Hiányzó kód mező	400
TestTOTPConfirm_ InvalidCode	Érvénytelen kód → secret törlés	401
TestTOTPDisable_ Success	Kikapcsolás érvényes kóddal	200
TestTOTPDisable_ NotEnabled	Kikapcsolás TOTP nélkül	400
TestTOTPDisable_ InvalidCode	Kikapcsolás rossz kóddal	401
TestTOTPDisable_ MissingCode	Hiányzó kód mező	400
TestTOTPVerifyLogin_ Success	Pending token + érvényes kód → JWT	200
TestTOTPVerifyLogin_ ExpiredToken	Lejárt vagy érvénytelen pending token	401
TestTOTPVerifyLogin_ InvalidCode	Érvénytelen TOTP kód	401

8.13. táblázat. A TOTP handler tesztek (részlet a 18-ból)

8.3.3. WebSocket tesztek

A WebSocket hub biztonsága és konkurens-korrektsége két külön tesztkészletet igényel:

- **JWT alapú upgrade-megtagadás.** Az új `pump_test.go` és `chat/handler_test.go` fájlok olyan eseteket fednek le, amelyek biztonsági szempontból kritikusak: lejárt JWT, hiányzó token és rossz aláírókulccsal aláírt token. Mindhárom eset 401 `Unauthorized` választ kap, és a WebSocket upgrade meg sem történik. Ezek az esetek korábban csak a teljes E2E tesztben voltak jelen; a külön handler-szintű tesztrel a hiba minél közelebb a bevezetés helyéhez (az upgrade-et végző handler kódjához) detektálható.

- **Hub konkurens hozzáférés.** A `TestHub_ConcurrentUnregisterAndOnlineUsers` eset 50 egymással párhuzamos `Unregister` és `OnlineUsers` hívást indít ugyanarra a Hub példányra. A teszt a `make test-race` céllal együtt futtatva a Go versenyállapot-detektorát [28] aktiválja, így a Hub belső `sync.RWMutex` használatának hibája (elfelejtett lock, dupla unlock, stb.) az első futáskor felszínre kerül.
- **writePump üzenetkézbesítés.** A `TestWritePump_DeliversMessageToClient` eset a teljes `writePump` goroutine-t végigfuttatja: a kliens kapcsolódik, a teszt egy üzenetet helyez a küldő csatornára, és a kliensoldal a valódi WebSocket frame-en megkapja azt. A `TestWritePump_ClosesOnChannelClose` a pump helyes leállítását igazolja, amikor a Hub bezárja a küldő csatornát.

Teszt eset	Leírás	Eredmény
TestServeWS_RejectsExpiredJWT	Lejárt JWT-vel dial	401
TestServeWS_RejectsMissingToken	Token nélkül dial	401
TestServeWS_RejectsWrongSecret	Rossz aláírókulccsal dial	401
TestWritePump_DeliversMessageToClient	Teljes írási goroutine-on át üzenet	Sikeres
TestWritePump_ClosesOnChannelClose	<code>Unregister</code> után a pump leáll	Sikeres
TestHub_ConcurrentUnregisterAndOnlineUsers	50 párhuzamos hívás, -race	Nincs race

8.14. táblázat. A WebSocket hub és handler integrációs tesztek

8.4. 3. réteg: End-to-end tesztek

A piramis csúcsán egyetlen tesztfájl (`internal/integration/api_test.go`) áll, amely a teljes rendszert egy valódi `httptest.Server` segítségével indítja el. Ugyanazt a `chi` routert, JWT middleware-t és WebSocket hubot használja, mint a `cmd/server/main.go`. Ezek a tesztek nem egyes komponenseket, hanem komplett felhasználói utakat (*user journey*) ellenőriznek: regisztráció → bejelentkezés → védett végpont, vagy két kliens közötti valós idejű üzenetváltás WebSocketsen.

A tesztkörnyezet kisméretű (64 bites) Schnorr-paramétereket alkalmaz a gyors végrehajtás érdekében: egy átlagos fejlesztői gépen a teljes E2E rétegre kb. fél másodperc kell.

Teszt eset	Leírás	Eredmény
Test_S4_1_GetSchnorrParams	Schnorr paraméterek lekérése	200
Test_S4_2_LoginInit_KnownUser	Ismert felhasználó login init	200
Test_S4_2_LoginInit_UnknownUser_Returns401	Ismeretlen felhasználó	401
Test_S4_3_ListUsers_Authenticated	Felhasználólista JWT-vel	200
Test_S4_4_GetMessageHistory	Üzenet-előzmények lekérése	200
Test_S4_4_GetMessageHistory_UnknownPeer_Returns404	Ismeretlen peer	404
Test_S4_5_TOTPStatus	TOTP státusz lekérdezés	200
Test_S4_6-TokenRefresh	Token frissítés	200
Test_S4_7_UnauthenticatedAccess_Returns401	Hitelesítés nélküli hozzáférés	401
Test_S4_7_MalformedToken_Returns401	Hibás token formátum	401
Test_S4_8_Register_InvalidVerifier_Returns400	Érvénytelen verifier	400
Test_S4_8_Register_MissingFields_Returns400	Hiányzó mezők	400
Test_S5_FullLoginFlow_ThenAccessProtectedEndpoint	Teljes login → védett végpont	Sikeres
Test_S5_DuplicateRegistration_Returns409	Duplikált regisztráció	409
Test_S5_RecoveryFlow	Teljes recovery folyamat	Sikeres
Test_S6_WebSocket_Connect	WebSocket kapcsolatfelvétel	Sikeres
Test_S6_WebSocket_InvalidToken	Érvénytelen tokennel WS	Elutasítva
Test_S6_WebSocket_NoToken	Token nélküli WS	Elutasítva
Test_S6_WebSocket_SendMessage	Üzenetküldés WebSocket-en	Sikeres
Test_S6_WebSocket_OfflineDelivery	Offline üzenet kézbesítés	Sikeres

8.15. táblázat. Az end-to-end tesztek

Egy negyedik, böngészőben futó UI tesztréteg (pl. *Playwright*-tel a JavaScript `BigInt` alapú crypto-kód végrehajtásának ellenőrzésére) technikailag lehetséges volna; a szakdolgozat keretében azonban a manuálisan elvégzett böngészős tesztet tartottam elégségesnek. A szerveroldali E2E már minden REST végpontot és WebSocket forgatókönyvet lefed, a maradék kockázat a kliens-oldali renderelés, amit a manuális forgatókönyvek hatékonyabban fognak meg.

8.5. Tesztelési összefoglaló

Csomag	Rétegek	Tesztek száma	Eredmény
<code>internal/crypto</code>	Unit	18	Mind sikeres
<code>internal/middleware</code>	Unit	8	Mind sikeres
<code>internal/chat</code> (hub, envelope)	Unit + Integration	7 + 5	Mind sikeres
<code>internal/service</code> (auth_unit, totp_unit)	Unit (mock)	7 + 14	Mind sikeres
<code>internal/service</code> (auth, totp, recovery)	Integration	15 + 16 + 11	Mind sikeres
<code>internal/repository</code>	Integration	11	Mind sikeres
<code>internal/handler</code> (auth, chat, totp)	Integration	26 + 10 + 18	Mind sikeres
<code>internal/integration</code> (api_test)	E2E	20	Mind sikeres
Összesen	3 réteg	186	Mind sikeres

8.16. táblázat. Tesztelési összefoglaló csomagonként és rétegenként

A három rétegre tagolt piramis gyakorlati előnye, hogy a fejlesztői visszajelzési ciklus drasztikusan lerövidül: a `make test-unit` a teljes egységréteget 1–2 másodperc alatt lefuttatja, így minden kódváltoztatás után automatikusan meghívható. Az integrációs és E2E rétegek csak a commit vagy CI lépéseknél indulnak el; ez utóbbi a `make test` szekvenciával teljes körűen lefut kb. 20 másodperc alatt.

9. fejezet

Összefoglalás

9.1. Elért eredmények

A dolgozat egy ZKP-alapú biztonságos csevegőalkalmazás tervezését és megvalósítását mutatta be. A Schnorr-protokollra épülő jelszómentes azonosítás, a TOTP kétfaktoros hitelesítés és a WebSocket-alapú valós idejű kommunikáció együttesen egy olyan rendszert alkotnak, amelyben a szerver soha nem fér hozzá a felhasználó titkos adataihoz.

A munka során a következő eredmények születtek:

- A Schnorr-protokoll gyakorlati alkalmazása egy teljes hitelesítési rendszerben, amely bizonyítottan soha nem teszi ki a felhasználó titkos kulcsát. A kliensoldali PBKDF2–SHA256 kulcsszármaztatás biztosítja, hogy a jelszó soha nem hagyja el a böngészőt.
- Egy hibrid kommunikációs architektúra megvalósítása, amely hatékonyan szétválasztja az állapotmentes hitelesítést (REST API) és az állapottartó valós idejű kommunikációt (WebSocket).
- A valós idejű chat funkció implementálása Hub architektúrával, jelenlétkelessel és offline üzenetkézbesítéssel.
- ZKP-alapú jelszóvisszaállítás recovery key segítségével, amely ugyanazt a Schnorr-protokollt alkalmazza, mint a normál bejelentkezés — így a kliens a recovery kulcsot sem fedi fel a szerver felé.

- Session-életciklus kezelés: 2 órás JWT érvényesség, automatikus token-frissítés, WebSocket heartbeat és lejárt tokenek automatikus lecsatlakoztatása.
- 77 egységteszt, amelyek teljes lefedettséget nyújtanak a kriptográfiai, hitelesítési, TOTP, recovery és üzenetkezelési rétegekre.

9.2. Korlátok

- A jelenlegi implementáció nem tartalmaz végpontok közötti titkosítást (E2EE), így a szerver hozzáfér az üzenetek tartalmához. Egy E2EE-réteg (pl. Signal Protocol) beépítése önálló tervezési feladat, amely a dolgozat hitelesítésre szűkített fókuszán kívül esik.
- Az SQLite adatbázis és a memóriabeli tárolók egyszerverű architektúrát feltételeznek, ami korlátozza a horizontális skálázhatóságot.
- A Docker konténerizálás a jövőbeli fejlesztés tárgyát képezi.
- A rendszer nem nyújt védeltséget kvantum-számítógépes támadásokkal szemben, mivel a diszkrét logaritmus probléma kvantum-környezetben hatékonyan megoldható (Shor-algoritmus).

9.3. Jövőbeli fejlesztési lehetőségek

- Végpontok közötti titkosítás (E2EE) bevezetése Signal Protocol vagy hasonló séma alkalmazásával.
- Docker konténerizálás a reprodukálható telepítés érdekében.
- PostgreSQL-re vagy más skálázható adatbázisra való áttérés, Redis bevezetése a horizontális skálázhatóság érdekében.
- Post-quantum kriptográfiai primitívek vizsgálata (pl. lattice-based ZKP sémák).
- Mobilalkalmazás fejlesztése (iOS, Android).

Irodalom

- [1] Shafi Goldwasser, Silvio Micali és Charles Rackoff. „The Knowledge Complexity of Interactive Proof-Systems”. *Proceedings of the 17th Annual ACM Symposium on Theory of Computing (STOC)*. 1985, 291–304. old. DOI: 10.1145/22145.22178.
- [2] Claus-Peter Schnorr. „Efficient Signature Generation by Smart Cards”. *Journal of Cryptology* 4.3 (1991), 161–174. old. DOI: 10.1007/BF00196725.
- [3] David M’Raihi és tsai. *RFC 6238: TOTP: Time-Based One-Time Password Algorithm*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6238>. 2011.
- [4] Douglas R. Stinson és Maura B. Paterson. *Cryptography: Theory and Practice*. 4. kiad. CRC Press, 2018. ISBN: 978-1-138-19701-5.
- [5] Victor Shoup. *A Computational Introduction to Number Theory and Algebra*. 2. kiad. Cambridge University Press, 2009. ISBN: 978-0-521-51644-0.
- [6] Kevin S. McCurley. „The Discrete Logarithm Problem”. *Proceedings of Symposia in Applied Mathematics* 42 (1990), 49–74. old.
- [7] Alfred J. Menezes, Paul C. van Oorschot és Scott A. Vanstone. *Handbook of Applied Cryptography*. Elérhető: <https://cacr.uwaterloo.ca/hac/>. CRC Press, 1996. ISBN: 978-0-849-38523-0.
- [8] Michael Jones, John Bradley és Nat Sakimura. *RFC 7519: JSON Web Token (JWT)*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc7519>. 2015.
- [9] Mohsen Toorani. „Security Protocols in a Nutshell”. *arXiv preprint* (2016). arXiv: 1605.09771v2.
- [10] Neha Upadhyay. „Cryptographic Foundations of Cloud Security: A Survey on Trust Models and Zero-Knowledge Proof-Based Authentication”. (2025).

- [11] Rohit Pathak és tsai. „Bibliometric Survey on Zero-Knowledge Proof for Authentication”. (2021).
- [12] Zhigang Chen és tsai. „A Survey on Zero-Knowledge Authentication for Internet of Things”. *Electronics* 12.5 (2023), 1145. old. DOI: 10.3390/electronics12051145.
- [13] Raghav Dwivedi és tsai. „Privacy Preserving Authentication System Based on Non-Interactive Zero Knowledge Proof Suitable for Internet of Things”. *Journal of Ambient Intelligence and Humanized Computing* (2021). DOI: 10.1007/s12652-021-03459-4.
- [14] Thiagarajan. „Enhancing Captive Portal Authentication with Zero-Knowledge Proofs”. (2024). SSRN: 5106822.
- [15] Kapil Saluja és Madhuri Bhatia. „Zero Knowledge Proof Based Authentication Protocol Using Graph Isomorphism”. (2019). arXiv: 1911.09329v1.
- [16] Ahmed Asimi és tsai. „Strong Zero-Knowledge Authentication Based on Virtual Passwords”. *International Journal of Network Security* 18.4 (2016), 601–616. old.
- [17] Feng Hao és tsai. *RFC 8236: J-PAKE: Password-Authenticated Key Exchange by Juggling*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc8236>. 2017. DOI: 10.17487/RFC8236.
- [18] Itzel Vazquez Sandoval és tsai. „PakeMail: Authentication and Key Management in Decentralized Secure Email and Messaging via PAKE”. (2021). arXiv: 2107.06090v1.
- [19] Kazukuni Kobara és Hideki Imai. „Pretty-Simple Password-Authenticated Key-Exchange Protocol”. (2001). arXiv: cs/0110024v1.
- [20] Mikhail Fomichev és tsai. „FastZIP: Faster and More Secure Zero-Interaction Pairing”. (2021). arXiv: 2106.04907v3. DOI: 10.1145/3458864.3467883.
- [21] Little és tsai. „Secure Account Recovery for a Privacy-Preserving Web Service”. (2024).
- [22] Faller és tsai. „Password-Protected Key Retrieval with(out) HSM Protection”. (2024). DOI: 10.1145/3658644.3690358.

- [23] Zhang és tsai. „Blocklisted Oblivious Pseudorandom Functions”. (2025). arXiv: 2507.16040v1.
- [24] Nguyen és tsai. „Zero-Knowledge Password Policy Check from Lattices”. (2018). arXiv: 1802.05004v1.
- [25] Gijs Van Laer és tsai. „Harden Zero Knowledge Password Proofs Against Offline Dictionary Attacks”. (2016).
- [26] Will Major és tsai. „An Authentication Protocol Based on Chaos and Zero Knowledge Proof”. *Nonlinear Dynamics* (2020). DOI: 10.1007/s11071-020-05463-3.
- [27] Povsic. „Zero-Knowledge Authentication”. (2022). arXiv: 2205.05847v1.
- [28] The Go Authors. *The Go Programming Language*. [https : / / go . dev](https://go.dev). Hozzáférés: 2026-02-27. 2024.
- [29] Gorilla Web Toolkit. *gorilla/websocket — A fast, well-tested and widely used WebSocket implementation for Go*. [https : / / github . com / gorilla / websocket](https://github.com/gorilla/websocket). v1.5.3. Hozzáférés: 2026-02-27. 2023.
- [30] Ian Fette és Alexey Melnikov. *RFC 6455: The WebSocket Protocol*. Internet Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc6455>. 2011.
- [31] go-chi. *chi — lightweight, idiomatic and composable router for building Go HTTP services*. <https://github.com/go-chi/chi>. v5.2.5. Hozzáférés: 2026-02-27. 2024.
- [32] SQLite Consortium. *SQLite — A C-language library that implements a small, fast, self-contained SQL database engine*. <https://www.sqlite.org>. v3.46. Hozzáférés: 2026-02-27. 2024.
- [33] modernc.org. *modernc.org/sqlite — CGo-free port of SQLite for Go*. [https : / / pkg . go . dev / modernc . org / sqlite](https://pkg.go.dev/modernc.org/sqlite). v1.46.1. Hozzáférés: 2026-02-27. 2024.
- [34] Paul Querna. *pquerna/otp — One Time Password utilities for Go*. [https : / / github . com / pquerna / otp](https://github.com/pquerna/otp). v1.5.0. Hozzáférés: 2026-02-27. 2024.
- [35] David M'Raihi és tsai. *RFC 4226: HOTP: An HMAC-Based One-Time Password Algorithm*. Internet Engineering Task Force. [https : / / www . rfc - editor . org / rfc / rfc4226](https://www.rfc-editor.org/rfc/rfc4226). 2005.

- [36] Ham Vocke. *The Practical Test Pyramid*. <https://martinfowler.com/articles/practical-test-pyramid.html>. martinfowler.com. 2018.
- [37] Mike Wacker. *Just Say No to More End-to-End Tests*. <https://testing.googleblog.com/2015/04/just-say-no-to-more-end-to-end-tests.html>. Google Testing Blog. 2015.
- [38] Google LLC. *Fundamentals of Testing Android Apps*. <https://developer.android.com/training/testing/fundamentals>. Letöltve: 2026-04-20.